Build with Parmana

Connect AI agents to real systems, with a person and a signed record behind every action.

Generated 2026-10-22

Build with Parmana

This book takes you from nothing to a production integration: an AI agent that can act on a real system only when a policy and a person allow it, with a signed record of every decision. It is the companion to The Parmana Handbook, which explains how each part of Parmana is built. This book explains how to **build with** it.

Download it as one PDF: build-with-parmana.pdf.

Who it is for

You are	Start with
A developer connecting an agent	Chapter 1, then Chapter 3
An operator bringing Parmana to production	Chapter 8, then Chapter 4 and Chapter 5
Connecting a system Parmana has no code for	Chapter 6
Reviewing Parmana for security	Chapter 10, then Chapter 7
An AI agent	Integrate Parmana: specification for AI agents, and `llms-full.txt` for this site in one file

How the book is written

- **Every statement is checked against the code.** Where the book and the code disagree, the code wins and the book is fixed. Anything not verified in production says so.
- **The reference chapters are generated from the code.** Every endpoint, every SDK method and every error code are produced by `scripts/generate-build-book-reference.ts` from the OpenAPI file, the SDK source and the server's error classes. A test fails when they fall behind the code.
- **Examples are real.** Responses are captured from the running server; tutorial output is what the tutorial prints.
- **Procedures state what to expect.** Where a step can fail, it says what you see and what to do.

The chapters

1. How Parmana works: one request, from an agent to a signed record.
2. Your first governed action: run it on your machine, step by step.
3. Connect an agent: the request, the policy in effect, approvals, in both SDKs.
4. Policies: write one, get it approved, change it, roll it back.
5. Human approvals: approvers, signing, and what an approval covers.
6. Connect your systems: built in connectors and external connectors.
7. Verify and audit: records you can check without trusting the server.
8. Deploy and operate: production, keys, migrations, monitoring.

9. Errors and troubleshooting: what each answer means and what to do.
10. Security model and limits: what is guaranteed, and what is not.

Reference: every endpoint, every SDK method, every error code.

Versions

The published SDKs are 1.6.0 (`@parmana/sdk` on npm, `parmana` on PyPI), published 2026-10-11. Methods marked **1.5.0** need that version or later: `policyInEffect` and `policy_in_effect`, `verifyParmanaRelease` and `verify_parmana_release`.

1. How Parmana works

Parmana sits between an AI agent and the systems it acts on. The agent never holds the credentials of those systems. It asks Parmana to perform an action, Parmana decides under a policy a person approved, and only an approved action is released to the system. Every decision, approved or refused, leaves a signed record anyone can check.

One request, in order

#	What happens	Where to read more
1	The agent asks which policy governs the action: `GET /policies/in-effect?capability=...`. The answer names the policy, its version in effect, and the signals a request must carry.	Chapter 3
2	The agent sends a **Business Transaction** to `POST /execute`: who is acting (authority), for what purpose (authorization), what to do (intent: action, target, parameters), the policy it declares, and its signals.	Chapter 3
3	Parmana checks the caller's key may use the action, and that the declared policy is the one bound to the action at the version in effect.	Policies and the decision
4	Signals bound to the request are checked against it, and an approval signal counts as true only with a valid signed approval from a trusted person for that action and resource.	Human approval
5	The policy's rules run in order; the first that matches decides. The same inputs always give the same decision.	Chapter 4
6	Refused: Parmana answers with the reason and stores a signed **Refusal Record**. Nothing is released.	Refusal Records
7	Approved: Parmana signs an **execution authorization**, single use, time bounded, and bound to the exact content approved.	Execution authorization
8	Before release it stores a signed **Execution Intent**: exactly what is about to be released.	Execution Intents
9	The **gateway** verifies the authorization again (signature, expiry, content, single use, the policy, the signals and the approval) and releases the action to the connector for that action.	The gateway
10	The connector acts with a credential only it can use, for this one execution.	Credential isolation
11	Parmana signs an **Execution Trust Record**: the transaction, the decision, the authorization and the connector's evidence.	Execution Trust Records
12	Anyone with the public key can verify the record offline, without trusting the server.	Chapter 7

The ideas that matter

- **No action without a person.** Every policy must require a signed human approval, reads included. A server refuses to load a policy that could approve without one. Plan for the first request on a new resource to be refused until a person signs.

- **The agent cannot authorize itself.** Facts the agent declares can refuse an action, never authorize one on their own. Only a signed approval, or a fact the server checks for itself, can make an approve rule match.
- **Parmana releases the action; the agent does not.** An agent that performs the action itself after asking Parmana defeats the control. The agent only reads the answer.
- **Changes need two people.** A new policy version, a new approver key and a new external connector each need one person to propose and a different person to approve with a step up signature. No deploy is needed.
- **Everything is evidence.** Approvals, refusals, intents and results are signed and kept. A refusal is as provable as an approval.

The words used in this book

Word	Meaning
Capability, action	What the agent wants done, `namespace:verb`, for example `paytm:refund`.
Business Transaction	One request to act, with its identifiers, intent, policy and signals.
Signal	A fact in the request that a policy reads, such as `refundAmount` or `managerApproved`.
Approval signal	A signal that counts as true only with a signed approval (`signals.approvalArtifact`).
Policy in effect	The version of the policy bound to an action that was most recently approved.
Maker, checker	The person who proposes a change, and the different person who approves it.
Connector	The part that performs an approved action on a system: built in (HubSpot, GitHub, Paytm, Slack) or external (your endpoint).

More in the glossary.

2. Your first governed action

This chapter runs everything on your machine, with no database and no cloud account. Every command and every line of

To see the same governance work on your own machine, follow the Quickstart: a request is refused without a signed approval, released with one, and its signed record verifies offline. This chapter then does the same on your own machine, where you also play the approver.

Before you begin

- Node.js 24.6 or later (ML-DSA-65 signing needs it), and Git.
- About ten minutes. `npm ci` and the first build take most of it.

1. Set up a fresh clone

```
# Repository access is granted by agreement with Parmana Systems.
cd parmana
cp.env.example.env
npm ci
npm run build
npx tsx scripts/generate-keypair.ts --algorithm ed25519 --key-id default
npm run generate:gateway-keys
```

```
# Repository access is granted by agreement with Parmana Systems.
cd parmana
Copy-Item.env.example.env
npm ci
npm run build
npx tsx scripts/generate-keypair.ts --algorithm ed25519 --key-id default
npm run generate:gateway-keys
```

What each step is for:

Step	Why
`.env` from `.env.example`	Only `.env.example` is in the repository. `.env` is ignored by Git and holds your local settings.
`npm run build`	Builds the OpenAPI bundle, every package and the TypeScript SDK the examples import.
The `default` key	Signs execution authorizations and records. `keys/` is ignored by Git, so a fresh clone has no key until you make one.
The `gateway` key	The gateway signs its own attestations with a separate key.

If a later step fails with a missing file under `keys/`, one of the two key commands was skipped.

2. Watch a person decide

Tutorial 119 runs the real `customer-refund` 1.2.0 policy and the components the server runs, in one process.

```
npx tsx examples/tutorials/119-human-approval-for-one-action/run.ts
```

It ends with:

```
1. A refund of 5000 with no manager: refused, every refund needs one (the agent's own
eligibility and fraud claims are not enough)
  REFUSED   reject-manager-approval-required: Refund rejected. Every refund needs a signed
manager approval for this order, covering this amount, in signals.approvalArtifact, with
managerApproved true, and every other policy condition satisfied.

2. A refund of 75000 with no manager: refused (the server also writes a signed Refusal Record a
manager can review)
  REFUSED   reject-manager-approval-required: Refund rejected....

3. The agent sends managerApproved: true with no approval: refused
  REFUSED   approval check: managerApproved=true != verified false

The manager reviews the refusal and signs: this order, up to 75000, 15 minutes, once.

4. The agent sends a new request with the signed approval: approved
  APPROVED  approve-refund-with-manager-approval

4b. The gateway checks the approval again just before release
  PASSED    release stage, approval verified, not consumed

5. The same approval sent again on another request: refused, an approval is used once
  REFUSED   approval check: managerApproved=true != verified false

6. An approval up to 75000 used for 90000: refused, the amount comes from the request
  REFUSED   approval check: managerApproved=true != verified false

7. An approval for another order: refused
  REFUSED   approval check: managerApproved=true != verified false

8. A refund of 150000, even with an approval: refused by the policy maximum
  REFUSED   reject-above-maximum: Refund rejected because the requested refund amount exceeds
the maximum of 100000, even with a manager approval.

✓ All 9 steps behaved as the policy and the approval rules require.
```

Read it as the four rules of Chapter 1 in action:

- The agent's own claims (eligible, fraud check passed) never approve a refund (steps 1 and 2).
- Saying `managerApproved: true` is worth nothing without a signed approval (step 3).
- A signed approval covers one action, one resource, up to one amount, once (steps 5 to 7).
- The policy still applies to an approved request (step 8).

3. Release an action to a system Parmana has no code for

Tutorial 123 registers an external connector through maker checker, runs an example endpoint from the SDK on your machine, and releases an approved request to it.

```
npx tsx examples/tutorials/123-external-connector/run.ts
```

The parts that matter (the server's own log lines are left out):

```
Step 1: register erp:create-invoice, maker then checker
Approved by checker  : 200 APPROVED
Registration         : active -> https://erp.example.com/parmana/release

Step 3: an approved request is released to the ERP
Release audience     : https://erp.example.com/parmana/release
Approved by          : manager-x
Signature            : ed25519, key default
Result success       : true
Invoices created     : INV-1

Step 4: Parmana sends the same release again
ERP answered         : 200, INV-1
Invoices created     : INV-1

Step 5: a release made for another endpoint
Another endpoint     : 401 release.audience "https://erp.example.com/parmana/release" is not
this endpoint (https://payroll.example.com/parmana/release)

Step 6: revoke the registration
Revoke approved      : 200 APPROVED
Next request         : CONNECTOR_NOT_REGISTERED
Invoices created     : INV-1

✓ Registered by two people, released signed, acted on once, refused elsewhere, stopped by a
revoke.
```

Chapter 6 explains each step.

4. Run the server and call it from the SDK

The tutorials run Parmana in one process. Now run the server and talk to it over HTTP, as an agent does.

Make a caller key for your agent

```
npx tsx scripts/generate-api-key.ts --caller-id local-agent --allowed-capabilities test:fixture-
execute
```

It prints the raw key once (`Key :...`) and the entry to configure (`{"callerId":"local-agent","keyHash":...}`). Only the hash is ever stored. Keep the raw key for the agent; put the entry in

`PARMANA_API_KEYS`.

Make an approver key and sign an approval

You play the approver here. On a real deployment the approver is a different person (Chapter 5).

```
npx tsx scripts/generate-approver-key.ts --approver-id local-test-approver --key-id local-test-approver-key-1 --out-dir./local-approver
npx tsx scripts/sign-approval.ts --private-key-file./local-approver/local-test-approver__local-test-approver-key-1.private.pem --approver-id local-test-approver --key-id local-test-approver-key-1 --capability test:fixture-execute --resource-id vendor/vendor-123 --max-amount 100 --out./local-approver/approval.json
```

The approval expires after 15 minutes unless you pass `--ttl-seconds`, and it can be used once.

Start the server

`NODE_ENV=test` with `PARMANA_STORAGE=memory` runs with no database. It also registers `test:fixture-execute`, a connector that needs no credential, and trusts the approver key in `PARMANA_TEST_APPROVER_PUBLIC_KEY_FILE`. The server refuses to start if that variable is set with any other `NODE_ENV`.

```
NODE_ENV=test \
PARMANA_STORAGE=memory \
PARMANA_API_KEYS='[{"callerId":"local-agent","keyHash":"<the hash printed above>","allowedCapabilities":["test:fixture-execute"]}]' \
PARMANA_TEST_APPROVER_PUBLIC_KEY_FILE=./local-approver/local-test-approver__local-test-approver-key-1.public.pem \
npm run dev
```

```
$env:NODE_ENV = "test"
$env:PARMANA_STORAGE = "memory"
$env:PARMANA_API_KEYS = '[{"callerId":"local-agent","keyHash":"<the hash printed above>","allowedCapabilities":["test:fixture-execute"]}]'
$env:PARMANA_TEST_APPROVER_PUBLIC_KEY_FILE = ".\local-approver\local-test-approver__local-test-approver-key-1.public.pem"
npm run dev
```

Paste the whole entry the key script printed, not a retyped one. In another terminal:

```
curl http://localhost:3000/health
# {"status":"UP"}

curl http://localhost:3000/version
# {"error":"authentication required"}

curl http://localhost:3000/version -H "Authorization: Bearer <the raw key>"
# {"name":"Parmana","version":"0.4.0","api":"v1"}
```

Every route except `/health`, `/ready` and the API description routes needs a key.

Send a request, refused, then approved

Save as `first-action.ts` in `typescript/examples/` (so `@parmana/sdk` resolves to the SDK you built) and run it with `PARMANA_API_KEY` set to the raw key:

```
import { readFileSync } from "node:fs";

import {
  createBusinessTransaction,
  ExecutionRejectedError,
  ParmanaClient,
} from "@parmana/sdk";

const client = new ParmanaClient({
  endpoint: "http://localhost:3000",
  apiKey: process.env.PARMANA_API_KEY!,
});

const me = await client.caller();
console.log(me);

const request = {
  principalId: me.callerId,
  purpose: "My first governed action",
  action: "test:fixture-execute",
  target: "vendor/vendor-123",
  parameters: { amount: 100, currency: "USD" },
  policy: { name: "vendor-payment", version: "2.1.0", schemaVersion: "1.0.0" },
};

const signals = {
  vendorVerified: true,
  invoiceVerified: true,
  paymentApproved: true,
  sufficientFunds: true,
  paymentAmount: 100,
  riskScore: 5,
  vendorId: "vendor/vendor-123", // bound: must equal target
};

try {
  await client.execute(
    createBusinessTransaction({
      ...request,
      signals: { ...signals, humanApproved: false },
    }),
  );
} catch (error) {
  if (!(error instanceof ExecutionRejectedError)) throw error;
  console.log("Refused:", error.message);
}

const approval = JSON.parse(
  readFileSync("../../local-approver/approval.json", "utf8"),
);
```

```
const record = await client.execute(
  createBusinessTransaction({
    ...request,
    signals: { ...signals, humanApproved: true, approvalArtifact: approval },
  }),
);

console.log(
  "Approved:",
  record.executions[0].decision.outcome,
  record.trustRecordHash,
);
```

```
cd typescript/examples
npx tsx first-action.ts
```

Output (your hash differs):

```
{ callerId: 'local-agent', allowedPrincipalIds: [ 'local-agent' ], allowedCapabilities: [
'test:fixture-execute' ], unrestrictedCapabilities: false }
Refused: Execution rejected: Vendor payment rejected because one or more required policy
conditions were not satisfied.
Approved: APPROVED c1ee106cb303c57723128efd46e4dba53fc6ec37f238aa410d9cfe54548120b5
```

`allowedPrincipalIds` defaults to the caller id when the key entry names none. That is why the request acts as `me.callerId`.

When something goes wrong

You see	Cause and fix
`401 {"error":"authentication required"}`	No key, a wrong key, or a hash in `PARMANA_API_KEYS` that is not the hash of that key.
The server refuses to start, naming `DATABASE_URL`	`NODE_ENV` is not `test`. Outside `test`, storage for replay protection and audit needs Postgres.
The server refuses to start, naming the approver file	`PARMANA_TEST_APPROVER_PUBLIC_KEY_FILE` is set while `NODE_ENV` is not `test`.
`Refused:... approval...` on the second request	The approval expired (15 minutes), was used already, or names another resource or a lower amount. Sign a new one.
`Cannot find module '@parmana/sdk'`	Run the script from `typescript/examples/`, after `npm run build`.

Next

Chapter 3 does the same from a real agent, against a real server, in both SDKs.

3. Connect an agent

An agent integrates with five calls. This chapter shows each in TypeScript and Python. The exact, literal procedure, with what to expect and what to do at every step, is Integrate Parmana: specification for AI agents; follow it when you build the agent.

Before you begin

- A Parmana server URL and an **API key issued for the agent**, with the action in its `allowedCapabilities`. The operator issues it (Chapter 8).
- The SDK: `npm install @parmana/sdk` or `pip install parmana`. For offline verification in Python, `pip install "parmana[verify]"`.
- An approver who can sign approvals for the action (Chapter 5).

1. Create a client and check the key

```
import { ParmanaClient } from "@parmana/sdk";

const client = new ParmanaClient({
  endpoint: process.env.PARMANA_URL!,
  apiKey: process.env.PARMANA_API_KEY!,
});

const me = await client.caller();
// { callerId: "my-agent", allowedPrincipalIds: ["my-agent"],
//   allowedCapabilities: ["paytm:refund"], unrestrictedCapabilities: false }
```

```
import os

from parmana import ParmanaClient

client = ParmanaClient(endpoint=os.environ["PARMANA_URL"],
api_key=os.environ["PARMANA_API_KEY"])

me = client.caller()
```

Your action must be in `allowedCapabilities`. The principal you act as must be in `allowedPrincipalIds`; the safe choice is your `callerId`. Never log or print the key.

2. Read the policy in effect (1.5.0)

Ask the server which policy governs the action, at which version, and what a request must carry. Never write a policy version into the agent: an approved new version replaces the old one at once, and a request naming an old one is refused.

```
const inEffect = await client.policyInEffect("paytm:refund");

inEffect.policy; // { name: "customer-refund", version: "1.2.0", schemaVersion: "1.0.0" }
inEffect.signals.facts; // ["fraudCheckPassed", "managerApproved", "refundAmount",
"refundEligible"]
inEffect.signals.approval; // { managerApproved: { resourceId: "parameters.orderId", value:
"parameters.amount" } }
```

```
in_effect = client.policy_in_effect("paytm:refund")

in_effect.policy.version        # "1.2.0"
in_effect.signals.facts         # ["fraudCheckPassed", "managerApproved", "refundAmount",
"refundEligible"]
in_effect.signals.approval      # {"managerApproved": {"resourceId": "parameters.orderId",
"value": "parameters.amount"}}
```

`signals.facts` is every fact the rules read. `signals.bound` names the signals that must equal a value of the request. `signals.approval` names the approval signal, and where in the request the resource (and amount) it approves are. With SDK 1.4.0 or earlier, call `GET /policies/in-effect?capability=...` directly; the answer is the same.

3. Build the request

`createBusinessTransaction` (TypeScript) and `create_business_transaction` (Python) fill in the identifiers the server checks against each other, so a request cannot fail on a mismatched id.

```
import { createBusinessTransaction } from "@parmana/sdk";

const transaction = createBusinessTransaction({
  principalId: me.callerId,
  purpose: "Refund order ORD-1042",
  action: "paytm:refund",
  target: "ORD-1042",
  parameters: { orderId: "ORD-1042", transactionId: "TXN-9", amount: 750 },
  policy: inEffect.policy,
  signals: {
    refundEligible: true,
    fraudCheckPassed: true,
    refundAmount: 750, // bound: must equal parameters.amount
    managerApproved: false, // true only with a signed approval, step 4
  },
});
```

```
from parmana import create_business_transaction

transaction = create_business_transaction(
    principal_id=me.caller_id,
    purpose="Refund order ORD-1042",
    action="paytm:refund",
    target="ORD-1042",
    parameters={"orderId": "ORD-1042", "transactionId": "TXN-9", "amount": 750},
```

```
    policy=in_effect.policy,
    signals={
        "refundEligible": True,
        "fraudCheckPassed": True,
        "refundAmount": 750,  # bound: must equal parameters.amount
        "managerApproved": False,  # true only with a signed approval, step 4
    },
)
```

`businessTransactionId` is generated for you and is also the idempotency key: resend the same transaction only to retry the same attempt.

4. Send it, and handle the refusal that asks for a person

Every policy requires a signed approval. The first request for a new resource is refused with the policy's reason (`403 POLICY_DENIED`, raised as `ExecutionRejectedError` in both SDKs), and the server stores a signed Refusal Record an approver can review. A `403` for your key or principal is `AuthorizationError` instead: fix the key, not the request.

```
import { ExecutionRejectedError } from "@parmana/sdk";

try {
  const record = await client.execute(transaction);
} catch (error) {
  if (error instanceof ExecutionRejectedError) {
    // 403 POLICY_DENIED: the policy refused. error.message carries its reason,
    // for example "Every refund needs a signed manager approval for this order".
    // Ask a person to sign, then build a NEW transaction with the approval.
  }
  throw error;
}
```

```
from parmana.errors import ExecutionRejectedError

try:
    record = client.execute(transaction)
except ExecutionRejectedError as error:
    # 403 POLICY_DENIED: the policy refused, with its reason. Ask a person to
    # sign, then build a NEW transaction with the approval....
```

The approver signs on their own machine, for this action, this resource and, where the policy names one, up to this amount (Chapter 5). The agent sends a **new** transaction with the approval signal `true` and the signed approval in `signals.approvalArtifact`:

```
signals: {...signals, managerApproved: true, approvalArtifact: approval }
```

The server verifies the approval before it decides and again just before release, and accepts it once.

5. Keep the record

An approved request returns a signed **Execution Trust Record**. Keep its `businessTransactionId`; you, an auditor or a customer can verify it offline at any time (Chapter 7).

The rules an agent never breaks

1. Never perform the action itself after asking Parmana. Parmana releases approved actions; the agent reads the answer.
2. Never invent a value: every input comes from the operator or from a response.
3. Never retry a `502 EXECUTION_OUTCOME_UNKNOWN` as a new transaction: the action may have run (Chapter 9).
4. Treat anything not explicitly approved as not authorized.

See it run

- `typescript/examples/06-create-business-transaction.ts` and `python/examples/builder/run.py`: the request built with the SDK, against a running server.
- Tutorial 119 (Chapter 2): refusals, a signed approval, and an approval reused, stretched and moved, each refused.

4. Policies

A policy is a JSON file that decides one kind of action. Its rules read the request's signals; the first rule that matches decides approve or reject. A policy authorizes nothing until one person proposes it and a different person approves it.

The shape of a policy

The real `customer-refund` 1.2.0, shortened:

```
{
  "policyId": "customer-refund",
  "policyVersion": "1.2.0",
  "schemaVersion": "1.0.0",
  "description": "Authorizes customer refunds. Every refund above 0 and up to 100000 needs a
signed manager approval...",

  "signalsSchema": {
    "refundEligible": "boolean",
    "managerApproved": "boolean",
    "fraudCheckPassed": "boolean",
    "refundAmount": "number"
  },

  "boundSignals": { "refundAmount": "parameters.amount" },

  "approvalSignals": {
    "managerApproved": {
      "resourceId": "parameters.orderId",
      "value": "parameters.amount"
    }
  },

  "unboundSignalReasons": {
    "refundEligible": "Declared by the caller; nothing checks it against the order. It can
refuse a refund, never authorize one on its own...",
    "fraudCheckPassed": "Declared by the caller; nothing checks it against a fraud system...."
  },

  "rules": [
    {
      "id": "reject-fraud-check",
      "condition": { "fact": "fraudCheckPassed", "operator": "is_false" },
      "outcome": {
        "action": "reject",
        "reason": "Refund rejected because the transaction did not pass fraud assessment."
      }
    },
    {
      "id": "reject-above-maximum",
      "condition": {
        "fact": "refundAmount",
        "operator": "gt",
```

```
        "value": 100000
      },
      "outcome": {
        "action": "reject",
        "reason": "Refund rejected because the requested refund amount exceeds the maximum of
100000, even with a manager approval."
      }
    },
    {
      "id": "approve-refund-with-manager-approval",
      "condition": {
        "all": [
          { "fact": "managerApproved", "operator": "is_true" },
          { "fact": "refundEligible", "operator": "is_true" },
          { "fact": "fraudCheckPassed", "operator": "is_true" },
          { "fact": "refundAmount", "operator": "gt", "value": 0 },
          { "fact": "refundAmount", "operator": "lte", "value": 100000 }
        ]
      },
      "outcome": {
        "action": "approve",
        "reason": "Refund authorized with a verified manager approval for this order and
amount."
      }
    },
    {
      "id": "reject-manager-approval-required",
      "condition": { "always": true },
      "outcome": {
        "action": "reject",
        "reason": "Refund rejected. Every refund needs a signed manager approval..."
      }
    }
  ]
}
```

The full file is `policies/customer-refund/1.2.0/policy.json`. A policy lives at `policies/<policyId>/<policyVersion>/policy.json`.

Field	Required	What it is
`policyId`	yes	The policy's name. Agents send it as `policy.name`.
`policyVersion`	yes	Its version. A change is a new version in a new folder, never an edit of an approved one.
`schemaVersion`	yes	The policy format, `1.0.0`.
`description`	no	What it allows, in words. Returned by `GET /policies/in-effect`.
`signalsSchema`	no	The type of each signal: `boolean`, `number` or `string`. Returned to agents as `signals.schema`. A request whose signal has another type is refused before any rule runs.
`boundSignals`	no	Signals that must equal a field of the request.

Field	Required	What it is
`approvalSignals`	no in the format, yes in practice	Signals that count as true only with a signed approval. An approve rule needs one.
`unboundSignalReasons`	no	Why a signal the rules read is neither bound nor an approval signal.
`rules`	yes	At least one, each with a unique `id`, a `condition` and an `outcome`.

Rules and conditions

Rules run in order and the **first match decides**. If none matches, the request is refused (`no_rule_matched`). End every policy with an `always` rule that rejects with a reason the agent can act on, as above.

An `outcome` is `{ "action": "approve" | "reject", "reason": "..." }`. The reason is returned to the agent and stored in the record, so write it for the person who reads it.

A `condition` is one of:

Condition	Matches when
`{ "fact": "x", "operator": "...", "value":... }`	The operator holds for signal `x`
`{ "all": [... ] }`	Every child matches
`{ "any": [... ] }`	At least one child matches
`{ "always": true }`	Always

The operators (`packages/policy/src/OperatorEvaluator.ts`):

Group	Operators
Equality	`eq`, `neq`
Numbers	`gt`, `gte`, `lt`, `lte`, `between`
Lists	`in`, `not_in` (value is a list); `contains`, `not_contains`, `contains_all`, `contains_any`
Text	`starts_with`, `ends_with`, `matches` (a regular expression)
Presence	`exists`, `not_exists`, `is_null`, `is_not_null`
Booleans	`is_true`, `is_false` (only the value `true` or `false`, never "truthy")
Length	`length_eq`, `length_gt`, `length_gte`, `length_lt`, `length_lte`

Every signal must be accountable

A signal is a claim. The policy says how each claim the rules read is held to account, or it does not load:

The signal is	Declare it in	What the server does
A copy of a request field	`boundSignals`	Refuses the request unless the signal equals that field (`"refundAmount": "parameters.amount"`).
A person's decision	`approvalSignals`	Treats it as `true` only with a valid signed approval for this action and resource (and amount).
Anything else	`unboundSignalReasons`	Nothing. The reason says why, so a reviewer sees the gap. Such a signal should only ever refuse.

An approval signal's `resourceId` is `"target"` or a path into `parameters`; its optional `value` is a path into `parameters` to a number the approval must cover. With more than one approval signal, give each its own `artifact` (the signal that carries its approval; the default is `approvalArtifact`).

The rule the validator enforces: no approval without a person

Every rule whose outcome is `approve` must require an approval signal with `is_true`, either as its whole condition or directly inside its top level `all`. A nested `any` does not count, because another branch could match. A policy that breaks this does not load, and every request under it is refused with:

```
Policy rule '<id>' approves without a signed human approval. Every approve rule must require an
approvalSignals fact
with is_true, as its whole condition or directly inside its top level 'all'.
```

The consequence for design: **the agent's own facts can refuse, only a person can approve.** Put the facts the agent declares in `unboundSignalReasons` and use them in reject rules and as extra conditions next to the approval.

Check a policy before proposing it

```
npx vitest run packages/policy/tests/unit/ReferencePolicies.test.ts
```

It loads every file under `policies/` through `PolicyValidator`, the same check the server runs. Common refusals:

Message	Fix
`Policy references fact(s) 'x' with no boundSignals entry and no unboundSignalReasons entry`	Bind it, make it an approval signal, or give a reason.
`approves without a signed human approval`	Add `{ "fact": "<approval signal>", "operator": "is_true" }` to the approve rule's top level `all`.
`unboundSignalReasons['x'] is contradictory`	A bound signal needs no reason. Remove one of the two entries.
`Duplicate policy rule id 'x'`	Rule ids are unique within a policy.

From a file to the policy in effect

A policy takes effect when a second person approves it. No deploy.

Step	Who	What
1	Author	Writes `policies/<name>/<version>/policy.json` and checks it.
2	Maker	`POST /policies/<name>/<version>/pending-changes` with `{ reason, proposedContent }`. Answer: `201`, `pendingPolicyChangeId`, `PENDING_APPROVAL`.
3	Checker	`GET /policies/pending-changes?status=PENDING_APPROVAL`, reads the content.
4	Checker	Signs a step up authorization for this change on their own machine and posts it to `.../<id>/approve` within 120 seconds.
5	Server	Checks the checker is a human, is not the maker, and the step up signature; writes a signed, chained approval record; saves the policy.
6	Everyone	From the next request, the new version is the one in effect for the action bound to the policy.

Maker and checker each need an API key registered as a human (`credentialHolderType USER`); the checker also needs a registered step up public key. The exact commands, in bash and PowerShell, with every answer and error, are in Policy lifecycle and approvals. The SDKs sign the step up authorization too: `signPolicyChangeStepUp()` and `sign_policy_change_step_up()`.

The same flow is in the SDKs as `proposePolicyChange`, `policyChanges`, `approvePolicyChange` and `rejectPolicyChange` (TypeScript), and `propose_policy_change`, `policy_changes`, `approve_policy_change` and `reject_policy_change` (Python) (SDK reference).

Change and roll back

- **Change.** Copy the folder to a new version, edit, check, propose, approve. Agents that read the policy in effect (Chapter 3) follow at once. A request naming the old version is refused before any rule runs, with a message naming the version in effect.
- **Roll back.** Propose and approve the older version again. The most recent approval decides the version in effect.
- **Send the file unchanged.** The approval record stores a hash of the approved content. CI's `verify-policy-approvals` job and the server at startup compare it with the file in Git; one changed character fails the check. `npx tsx scripts/verify-policy-changes-approved.ts --full-scan` runs the same comparison.

What still needs a deploy: binding a **built in** action to a different policy name. A new action for your own system needs neither: register it as an external connector, with its policy, through maker checker (Chapter 6).

What is checked on every request

- The policy named is the one bound to the action, at the version in effect.

- The policy has an approval record, the record's signature verifies, and its content hash equals the live content. A policy edited in the database outside this flow authorizes nothing.
- The gateway checks the approval record again just before release.

These checks are always on in production. In `NODE_ENV` `test` or `development` they are off unless `POLICY_EXECUTION_VERIFICATION_ENFORCED=true`, which is why the local server in Chapter 2 accepts unapproved example policies.

See it run

- Tutorial 119: the real `customer-refund` 1.2.0, each rule in turn.
- `examples/tutorials/14-custom-policy`: a custom policy, approved and refused.
- Write your first policy: a policy from scratch, step by step.

5. Human approvals

No agent action runs without a person. A policy names an **approval signal**; the agent may set it to `true` only with a **signed approval** from a trusted approver, for this action and this resource (and amount, where the policy names one), not expired, used once. This chapter is about the people and the signatures.

Three kinds of people, three kinds of keys

Person	Signs	Key	Registered through
Action approver	One action on one resource: "this refund, once"	An Ed25519 approver key, on their own machine	Manage approvers, maker checker
Maker	Nothing; proposes changes	An API key registered as a human	The operator (Chapter 8)
Checker	Approval of a change: a policy, an approver, a connector	An API key registered as a human, and a step up key	The operator

Private keys never leave their owner's machine. Only public keys are sent to anyone.

Trust an approver

An approver is trusted when their public key is in the server's approver list. Add one through maker checker: no code change, no deploy.

1. The approver makes a key pair on their own machine:

   ```
   npx tsx scripts/generate-approver-key.ts --approver-id approver-example --key-id approver-example-key-1 --out-dir ~/.parmana
   ```

   They keep `approver-example__approver-example-key-1.private.pem` and send the `.public.pem` file.

2. A maker proposes adding it: `POST /approval-issuers/changes` with `action: "add"`, the ids, the public key and a reason; or `proposeApproverChange()` in TypeScript, `approvers.propose_add()` in Python.

3. A checker compares the public key with the approver over another channel, signs a step up authorization for the change, and approves: `POST /approval-issuers/changes/<changeId>/approve`.

4. From the next request on, approvals signed with that key verify. `GET /approval-issuers` lists every trusted key and where it came from (`governed`, or `code` for keys listed in the server's source).

Rotate by adding a new key id, switching, then revoking the old one. Revoking refuses every approval the key ever signed, used or not. Key ids are never reused. Every command, in curl and both SDKs, is in Manage approvers.

Sign an approval

The approver signs on their own machine, for one action, one resource and, where the policy names a value, a maximum.

```
npx tsx scripts/sign-approval.ts \
  --private-key-file ~/.parmana/approver-example__approver-example-key-1.private.pem \
  --approver-id approver-example --key-id approver-example-key-1 \
  --capability paytm:refund --resource-id ORD-1042 --max-amount 75000 --out approval.json
```

```
import { readFileSync } from "node:fs";
import { signApproval } from "@parmana/sdk";

const approval = signApproval({
  privateKeyPem: readFileSync(
    "approver-example__approver-example-key-1.private.pem",
    "utf8",
  ),
  approverId: "approver-example",
  keyId: "approver-example-key-1",
  capability: "paytm:refund",
  resourceId: "ORD-1042",
  maxAmount: 75000,
});
```

```
from pathlib import Path

from parmana.crypto import sign_approval

approval = sign_approval(
    private_key_pem=Path("approver-example__approver-example-key-1.private.pem").read_text(),
    approver_id="approver-example",
    key_id="approver-example-key-1",
    capability="paytm:refund",
    resource_id="ORD-1042",
    max_amount=75000,
)
```

A signed approval, as produced in Chapter 2:

```
{
  "payload": {
    "version": 1,
    "approvalId": "89e8fff3-7a67-4a54-be9b-c2954f9321d5",
    "issuer": {
      "approverId": "local-test-approver",
      "keyId": "local-test-approver-key-1"
    },
    "issuedAt": "2026-10-01T01:15:46.863Z",
    "expiresAt": "2026-10-01T01:30:46.863Z",
    "capability": "test:fixture-execute",
    "resourceId": "vendor/vendor-123",
    "scope": { "field": "value", "comparator": "lte", "value": 100 },
```

```
    "nonce": "d796e70d-63c5-48ac-9e2e-6c3a369b32d5"
  },
  "signature": {
    "algorithm": "ed25519",
    "keyId": "local-test-approver-key-1",
    "value":
"d1HjmV295NmvPKjQS9nalL3hjvxE+SvoQ3WjTGcaLyO2Z6dFOwTQIENtGCAwrHqS2L2mqPxx2mnckrwE3WoyDA==",
    "signedAt": "2026-10-01T01:15:46.863Z"
  }
}
```

It lasts 15 minutes unless the approver sets `--ttl-seconds` (`ttlSeconds`, `ttl_seconds`). The signing tools refuse more than one day; the server checks only that it has not expired, so a short lifetime is the approver's choice to keep.

What the server checks

The agent sends a new request with the approval signal `true` and the approval in `signals.approvalArtifact` (or the signal the policy names as the declaration's `artifact`). Before deciding, and again at the gateway just before release, the server checks:

Check	If it fails
The approver and key are trusted and not revoked	The approval signal counts as `false`; the request is refused
The Ed25519 signature over the payload verifies	Same
`expiresAt` is in the future	Same
`capability` is the request's action	Same
`resourceId` equals the request field the policy names	Same
The amount in the request (never the agent's signal) is within `scope`	Same
`scope.field` is `value` (the policy gives a `value`) or `resourceId`	Same
The approval was not used before (its nonce is stored once used)	Same

A refused approval is a `403 POLICY_DENIED`, because the declared signal does not match what the server verified:

```
Rejected: declared signal(s) do not match independently verified state (managerApproved=true !=
verified managerApproved=false).
```

How a person learns an approval is needed

A request refused for want of an approval is a normal, expected answer. Three ways to reach the approver:

- **Refusal Records.** Every refusal is stored, signed, with its reason and the request. Query `refusal_records` for the policy and `matchedRuleId` (Human approval).
- **A webhook.** With `APPROVAL_WEBHOOK_URL` and `APPROVAL_WEBHOOK_SECRET` set, Parmana posts an `approval.needed` event, signed with HMAC SHA256, only when the same request with the approval signals `true` would have been approved. Its `approvals` list is exactly what to sign.
- **Email.** With Resend configured and `APPROVAL_EMAIL_TO` set, the same event is emailed, at most once per refusal.

Delivery of the webhook and the email is best effort: one attempt, no retry, and a failure never changes the refusal. The Refusal Records stay the complete list. Setup and receiver code: Approval notifications.

What an approval does not fix

- **A refused request is never approved later.** Nothing waits. After the approver signs, the agent sends a new request.
- **An approval covers the action and the resource, and the amount where the policy names one.** It does not fix other parameters: an approval for a Slack channel does not fix the message text. The approver trusts the agent with those details for that one use.
- **Facts the agent declares stay its word.** A policy should use them only to refuse.
- **There is no automatic path.** No policy can approve without a person; the validator refuses to load one that tries (Chapter 4).

Approving changes: the step up authorization

Approving a policy, an approver or an external connector is a different signature: a **step up authorization** from the checker, for one change and one decision (`approve` or `reject`), valid for at most 120 seconds and once. The server also refuses it when the checker is the maker (`403 SAME_ACTOR_CANNOT_APPROVE_OWN_CHANGE`) or not registered as a human (`403 NON_HUMAN_CALLER_DENIED`).

```
npx tsx scripts/sign-policy-change-step-up.ts --private-key-file step-up.private.pem \
  --key-id <checker caller id> --pending-policy-change-id <change id> --action approve
```

```
import { signPolicyChangeStepUp } from "@parmana/sdk";

const stepUp = signPolicyChangeStepUp({
  pendingPolicyChangeId: changeId,
  action: "approve",
  privateKeyPem: readFileSync("step-up.private.pem", "utf8"),
  keyId: "checker-step-up-1",
});
```

```
from parmana.crypto import sign_policy_change_step_up

step_up = sign_policy_change_step_up(
    pending_policy_change_id=change_id,
    action="approve",
```

```
    private_key_pem=Path("step-up.private.pem").read_text(),
    key_id="checker-step-up-1",
)
```

`scripts/local-review-action.ts` signs and submits in one process, so the 120 seconds cannot run out between the two.

Keep it honest

Parmana proves that a key signed. It cannot prove whose hand held the key. Two keys held by one person make every record claim a second person who does not exist. Keep each key on its owner's machine, and never paste a key into a chat or a ticket.

See it run

- Tutorial 119: an approval accepted, then refused when reused, stretched and moved.
- Tutorials 120 to 122 in `examples/tutorials/`: approvals for the rule itself, a read, a Slack post and a HubSpot update.

6. Connect your systems

Parmana never gives the agent a system's credentials. An approved action is released by the gateway to a **connector**, and only the connector touches the system. There are two kinds.

Kind	For	Credentials held by	Added by
Built in	HubSpot, GitHub, Paytm, Slack	Parmana's server, one per execution	Code in Parmana, configured by the operator
External	Any system with an HTTPS endpoint you run	Your endpoint, never Parmana	Two people, through maker checker, no deploy

Built in connectors

Each is registered only when its configuration is present (`packages/api/src/bootstrap/createConnectorRegistry.ts`).

Capability	Bound policy	Configuration
`hubspot:deal-fetch`	`hubspot-deal-read`	`HUBSPOT_PRIVATE_APP_TOKEN`
`hubspot:deal-update`	`hubspot-deal-update`	`HUBSPOT_PRIVATE_APP_TOKEN`
`github:pr-fetch`	`github-pr-read`	`GITHUB_APP_ID`, `GITHUB_INSTALLATION_ID`, `GITHUB_APP_PRIVATE_KEY`
`github:pr-merge`	`github-pr-approval`	The same
`paytm:refund`	`customer-refund`	`PAYTM_CONNECTOR_URL`, `PAYTM_CONNECTOR_SHARED_SECRET`
`slack:post-message`	`slack-post-message`	`SLACK_BOT_TOKEN`, `SLACK_ALLOWED_CHANNEL_IDS`

A capability whose connector is not configured is refused with `503 CONNECTOR_NOT_REGISTERED`, before anything is sent. Ask the server which policy and version govern a capability with `GET /policies/in-effect?capability=...`.

Adding another built in connector is a change to Parmana's code in several packages (the connector, the gateway adapter, the catalog, credentials, signal verification, the policy binding). For a system of your own, use an external connector instead.

External connectors

You run a small HTTPS endpoint in front of your system. Parmana releases every approved request for one capability to it as a **signed release**. The endpoint verifies the release with an SDK helper, acts with its own credentials, and answers. No shared secret is involved: the signature is the authentication.

What the endpoint receives

```
{
  "release": {
    "version": 1,
    "connectorId": "ext-erp:create-invoice",
    "audience": "https://erp.example.com/parmana/release",
    "businessTransactionId": "…",
    "authorizationId": "…",
    "capability": "erp:create-invoice",
    "target": "customer-42",
    "parameters": { "amount": 1200, "currency": "EUR" },
    "policy": { "name": "erp-invoice", "version": "1.0.0", "contentHash": "…" },
    "approvedBy": [
      {
        "approverId": "manager-x",
        "keyId": "manager-x-key-1",
        "approvalId": "…"
      }
    ],
    "issuedAt": "2026-10-01T10:00:00.000Z",
    "expiresAt": "2026-10-01T10:01:00.000Z"
  },
  "signature": { "algorithm": "ed25519", "keyId": "default", "value": "…" }
}
```

`parameters` holds only the names the registration allows. The release expires 60 seconds after it is issued.

What the endpoint does

The SDK helper does the checks; your code does the action.

```
import { verifyParmanaRelease } from "@parmana/sdk"; // 1.5.0

const check = await verifyParmanaRelease(body, {
  publicKeys: { default: parmanaPublicKeyPem }, // GET /keys/default
  audience: "https://erp.example.com/parmana/release", // exactly as registered
  isAlreadyExecuted: (businessTransactionId) =>
    store.has(businessTransactionId),
});
// check.valid false: answer 401 with check.errors, do nothing.
// check.alreadyExecuted true: answer with the stored result, do not act again.
```

```
from parmana.crypto import verify_parmana_release  # 1.5.0

check = verify_parmana_release(
    body,
    public_keys={"default": parmana_public_key_pem},
    audience="https://erp.example.com/parmana/release",
    is_already_executed=lambda business_transaction_id: business_transaction_id in store,
)
```

```
# check.valid False: answer 401 with check.errors, do nothing.
# check.already_executed True: answer with the stored result, do not act again.
```

The helper checks, in order: the body's shape, the Ed25519 signature over the canonical release with the key named in `signature.keyId`, that `audience` is this endpoint, and that the release has not expired (with 30 seconds of clock skew). The complete, runnable endpoints are `typescript/examples/07-external-connector-endpoint.ts` and `python/examples/13_external_connector_endpoint.py`; replace their `act` function with the call into your system.

Then the endpoint:

1. Acts only on `capability`, `target` and `parameters` from the verified release.
2. Records its answer by `businessTransactionId` in durable storage. Parmana may send the same release again after a timeout; the endpoint answers with its first result and does not act twice.
3. Answers `200` with `{ businessTransactionId, capability, success, result, executedAt }`, echoing the first two from the release. `result` is an object of at most 16 KB.

Register it: two people, no deploy

Step	Who	What
1	Author	Writes the capability's policy, with an approval signal (Chapter 4).
2	Maker, checker	Propose and approve the policy.
3	You	Deploy the endpoint over HTTPS. `POST` with body `{}` must answer `401`.
4	Maker	`POST /external-connectors/changes` with `action: "register"`, `capability`, `endpointUrl`, `policy`, `allowedParameters`, optional `timeoutMs` (1000 to 30000, default 10000), and a `reason`.
5	Checker	Reviews and approves with a step up authorization: `POST /external-connectors/changes/<changeId>/approve`.
6	Operator	Adds the capability to the agent key's `allowedCapabilities`.
7	Anyone	`GET /policies/in-effect?capability=...` names the policy and its approval signal.

From then on the agent integrates exactly as in Chapter 3; nothing differs for an external connector. The literal procedure, with what to expect and what to do at each step, is Connect any external system.

Rules Parmana enforces

Rule	Checked
The capability is `namespace:verb` and not in a built in namespace (`paytm`, `hubspot`, `github`, `slack`, `test`)	At proposal
The endpoint is `https`, names a host (no IP address, no `localhost`), and resolves only to public addresses	At proposal, at approval and at every release
Parmana connects to the address it checked and follows no redirect	At every release
Only the allowed parameter names are forwarded; a request naming another is not sent	At every release

Rule	Checked
One active registration per capability; a revoke stops releases at once	Always
Nothing is deleted; a revoked registration stays listed	Always

When it goes wrong

Answer	Meaning
`400 EXTERNAL_ENDPOINT_ADDRESS_REFUSED`	At proposal or approval: the host is not `https`, is an IP address or `localhost`, or resolves to an address that is not public.
`409 CONFLICT`	The capability already has an active registration or a pending change.
`503 CONNECTOR_NOT_REGISTERED`	No active registration. Nothing was sent.
`502 EXECUTION_OUTCOME_UNKNOWN`	The release failed and the endpoint **may have acted**: it answered something other than `200`, timed out, or answered without the echoed fields. Never resend as a new transaction (Chapter 9).

Status

Registration through maker checker, and releasing approved requests to the registered endpoint, are live in An approved request took 24 to 32 seconds there: give agents a timeout of at least 120 seconds. `verifyParmanaRelease` and `verify_parmana_release` are in SDK 1.5.0 and later.

See it run

Tutorial 123 (Chapter 2) registers an ERP endpoint through maker checker, releases to the TypeScript example endpoint, answers a retry once, refuses a release made for another endpoint, and stops at a revoke. The design and its security reasoning are in.

7. Verify and audit

Every decision Parmana makes leaves signed evidence. An auditor, a customer or your own agent can check it with Parmana's public key alone, with no account and no running server.

The records

Record	Written when	Proves	Read it with
Execution Trust Record	An approved action was released and answered	The transaction, the decision, the authorization, and the connector's evidence, signed	`GET /trust-records/{businessTransactionId}`, `trustRecord()`, `trust_record()`
Refusal Record	A policy refused a request	The request and the refusal, with its reason and the policy content hash, signed	`GET /refusal/{businessTransactionId}`, `refusalRecord()`, `refusal_record()`
Execution Intent	Just before an approved action is released	Exactly what was about to be released	`GET /execution-intents/{businessTransactionId}`, `executionIntent()`
Policy approval record	A checker approved a policy version	Who proposed, who approved, the content hash before and after, chained to the previous one	`policy_change_approval_records` table
Caller audit events	A request was refused before any policy ran	Who called, for what, and why it was refused	`caller_audit_events` table

The records answer different questions. A Trust Record says "this ran, under this decision". A Refusal Record says "this was asked and refused". An Execution Intent says "this was about to run", which matters when the run's outcome is unknown.

Check a Trust Record offline

Get Parmana's public key once (`GET /keys/default`, or `client.publicKey("default")`), keep it, and check any record against it.

```
import { readFileSync } from "node:fs";
import { verifyExecutionTrustRecordOffline } from "@parmana/sdk";

const record = JSON.parse(readFileSync("trust-record.json", "utf8"));
const publicKeyPem = readFileSync("parmana-default.public.pem", "utf8");

const result = verifyExecutionTrustRecordOffline(record, {
  default: publicKeyPem,
});
// { valid, hashValid, legacySignatureValid, hybridSignaturesValid?, algorithmsChecked, errors }
```

```
import json
from pathlib import Path

from parmana.crypto import verify_execution_trust_record_offline  # pip install
"parmana[verify]"

record = json.loads(Path("trust-record.json").read_text())
public_key_pem = Path("parmana-default.public.pem").read_text()

result = verify_execution_trust_record_offline(record, {"default": public_key_pem})
```

`valid` is true only when the record's hash equals a fresh SHA-256 of its canonical form **and** the Ed25519 signature verifies. Change any field, however deep, and both fail. `verifyExecutionIntentOffline` and `verify_execution_intent_offline` do the same for an Execution Intent. The Python verifier is a separate implementation, not a wrapper; `python/tests/test_offline_verifier.py` checks it against records signed by the TypeScript signer.

Three things to know when you build your own verifier:

- **Canonical form.** Keys are sorted; the bytes must be exactly what Parmana signed. Serializing the JSON again through a tool that reorders keys or reformats timestamps breaks verification of an untampered record. Keep the JSON exactly as received.
- **Large records.** A record over 4096 bytes of canonical content may be signed by AWS KMS over a fixed 97 byte commitment: the bytes of `PARMANA-ED25519-LARGE-MESSAGE-V1`, one NUL byte, then the SHA-512 digest of the canonical bytes. Try the raw signature first, and the commitment only for a record over 4096 bytes. The SDK functions do this.
- **Hybrid signatures.** A record may also carry ML-DSA-65 signatures in `signatures`. The TypeScript verifier checks them; the Python verifier checks Ed25519 only.

The CLI `scripts/verify-trust-record.ts` does the same from a terminal. Verify independently walks through it, with a tampered record failing; Detect tampering shows parameter tampering, a forged signature and a reused nonce, each caught.

Check with the server

`verify(businessTransactionId)` asks the server to verify a record again and appends the result to the record's history; `getLatestVerification()` and `get_latest_verification()` read the last one. These trust the server. The offline functions do not: prefer them when the question is whether to trust the server.

What a record proves, and what it does not

A valid Trust Record proves	It does not prove
Parmana's key signed exactly this content	That the key was not misused by whoever held it
The request, the decision and the policy version, as Parmana saw them	That facts the agent declared were true

A valid Trust Record proves	It does not prove
The connector's evidence as the connector reported it	That the system actually did it: an external endpoint's answer is its claim
The approval's issuer, as a key that verified	Whose hand held the approver's key

Chapter 10 lists every limit.

When the outcome is unknown

If the release fails after it may have reached the system (a timeout, a crash, an error from the connector), the agent gets `502 EXECUTION_OUTCOME_UNKNOWN` and the Execution Intent stays open. **The action may have run.** Never send it again as a new transaction.

An operator closes it:

1. **List** what needs attention: `GET /execution-intents/unfinalized` (or `unfinalizedExecutionIntents()`). Each entry has the signed intent and its state.
2. **Decide by state.**

State	Meaning	Do
`RELEASED`	The connector answered; the Trust Record was not stored	`POST /execution-intents/{id}/finalize`: rebuilds the record without calling the connector. Idempotent.
`PREPARED`	Stored, and the release may or may not have happened	Check the system, then resolve
`ERRORED`	The release raised an error; the action may still have run	Check the system, then resolve

3. **Resolve** after checking the system with the intent's `businessTransactionId`, `action` and `target`: `POST /execution-intents/{id}/resolve` with `{ "resolution": "NOT_EXECUTED" | "EXECUTED", "note": "what you checked" }`. The note is required. `resolvedBy` comes from your key.

A resolution is a person's attributed statement in the intent's unsigned status. It is a trail, not tamper evident evidence. The full procedure, with every answer: Execution Intents.

Audit queries

Refunds refused, with their reasons:

```
select refusal_record_id, business_transaction_id, submitted_by, created_at,
       decision_json ->> 'reason' as reason
from refusal_records
where decision_json -> 'policy' ->> 'name' = 'customer-refund'
order by created_at desc
limit 50;
```

Requests refused before any policy ran:

```
select occurred_at, caller_id, capability, type, reason
from caller_audit_events
where type in ('caller.rejected', 'caller.capability_denied', 'caller.principal_denied',
               'caller.non_human_denied', 'caller.structural_rejected')
order by occurred_at desc
limit 50;
```

A Refusal Record is written after the refusal, and a failed write does not change the refusal; each miss is logged as `refusal_record_write_failed`. So a query can miss a refused request, and the logs are the backstop.

Policy approvals: `npx tsx scripts/verify-policy-changes-approved.ts --full-scan` compares every policy on disk with its latest signed approval record.

8. Deploy and operate

This chapter is the operator's map. Each step has a check; do not go on until it passes. The complete runbook, with every command and every failure, is Production deployment; every variable is in the Environment variable reference.

What runs

One Node.js process: the API, the policy engine and the gateway together. The same code runs as a container (`Dockerfile` , `docker/entrypoint.sh`) or as a Vercel function (`api/index.ts` , `vercel.json`). It needs a Postgres database (Supabase works). Production refuses to start without a real database, signing keys and caller authentication, and never falls back to something weaker.

The procedure

#	Step	Do	Check
1	Signing keys	`npx tsx scripts/generate-keypair.ts --algorithm ed25519 --key-id default` and `npm run generate:gateway-keys` , or `KEY_PROVIDER=aws-kms`	`keys/` holds `default` and `gateway` , private and public. Never committed, never in the image.
2	Database schema	`npm run db:migrate -- status` , then `apply`	`status` ends with `0 pending` .
3	Caller keys	`npm run generate:api-key -- --caller-id <id>...` for each agent, a maker (`--credential-holder-type USER`) and a checker (also `--generate-step-up-key`)	Each raw key given to its owner only; the JSON entries combined into one array.
4	Connectors	Set the variables of each built in connector you use (Chapter 6)	Both or neither of each pair is set.
5	Environment	`NODE_ENV=production` , `PARMANA_API_KEYS` , `PARMANA_STORAGE=supabase` , `DATABASE_URL` (the transaction pooler on serverless), keys through `PARMANA_KEY_DIR` or `PARMANA_KEY_MATERIAL_JSON`	`PARMANA_AUTH_DISABLED` is not set; `NODE_ENV` is not `test` or `development` .
6	Deploy	Build and start, or deploy on Vercel	`/health` `200` ; `/ready` `200` with `"authDisabled":false` ; `POST /execute` with no key `401` .
7	Policies	Copy them into the database once (`scripts/apply-policies-table-and-backfill.ts`), then approve each one through maker checker (Chapter 4)	`npx tsx scripts/verify-policy-changes-approved.ts --full-scan` passes.
8	Approvers	Trust at least one approver per action through maker checker (Chapter 5)	`GET /approval-issuers` lists them, `"revoked": false` .
9	One real request	A small, harmless action, with a signed approval	A signed Trust Record; the same `businessTransactionId` again answers `409` ; the record verifies offline.
10	Monitoring	The alerts below	Each alert fires in a test.

At startup the log line `runtime_engine_constructed` must show `policyExecutionVerifierConfigured`, `signingReadinessConfigured` and `executionIntentsConfigured` all `true`. In production they are on and cannot be switched off.

Keys and who holds them

Key	Held by	Where it is configured
Signing key `default`	The server	`PARMANA_KEY_DIR`, `PARMANA_KEY_MATERIAL_JSON`, or AWS KMS
Gateway key	The server	The same, as a file
Caller API keys	Each agent and each person	Only the SHA-256 hash, in `PARMANA_API_KEYS`
Step up keys	Each checker, on their machine	The public key, on the checker's API key entry
Approver keys	Each approver, on their machine	The public key, through `/approval-issuers` maker checker
Connector credentials	The server, per execution	Environment variables or AWS Secrets Manager (`PARMANA_SECRETS_PROVIDER`)
External system credentials	Your endpoint	Never in Parmana

A caller key grants its `allowedCapabilities` and acts for its `allowedPrincipalIds` (the caller id when none are named). Never grant `"*"` to an agent.

Upgrades

1. **Migrations first.** `npm run db:migrate -- apply` against production **before** deploying the version that needs them. A version that finds a table missing refuses requests (for example `503 EXECUTION_INTENT_UNAVAILABLE`) rather than run without it.
2. **New policy versions are not deploys.** They take effect when approved (Chapter 4).
3. **A changed environment variable needs a new deployment** on Vercel before it takes effect.
4. **Restarts are safe.** On `SIGTERM` in flight requests finish (up to `SHUTDOWN_TIMEOUT_MS`, default 10000), and the replay and audit stores are in the database, not in memory.

Monitoring

Watch	Alert when
`GET /ready`, every 30 seconds	Not `200`, or `authDisabled` is `true`
`503 SIGNING_UNAVAILABLE`, `503 EXECUTION_INTENT_UNAVAILABLE`	Any. Nothing is being released.
`500 EXECUTION_RECORD_INCOMPLETE`	Any. Released with no signed record: finalize it (Chapter 7).

Watch	Alert when
`502 EXECUTION_OUTCOME_UNKNOWN`	Any. Check the system and resolve the intent. Several in a row: the connector is down.
`GET /execution-intents/unfinalized`, every 15 minutes	Any entry
Log `execution_intent_*_failed`	Any. Logged at critical severity.
Log `approval_issuer_lookup_failed`	Any. Approvals from governed approvers are refused until the table can be read.
Log `rate_limit_store_not_durable`	Any. Rate limits are per instance: `DATABASE_URL` is missing.
Log `approval_issuer_change_approved`, `external_connector_change_approved`	Every one, as an audit trail of who changed trust.

Rotation

Rotate	How
A caller key	Generate a new entry, replace the old one in `PARMANA_API_KEYS`, redeploy, give the owner the new raw key.
The signing key	`npx tsx scripts/rotate-verification-key.ts --algorithm ed25519`. Keep the old public key: old records name their key id.
An approver key	Add the new key id, switch, revoke the old one, all through maker checker. No deploy.
A connector secret	Change it on the host or in Secrets Manager, then redeploy. Values are read at startup.

A key that was ever shown in a chat, a ticket or a log is exposed: rotate it.

Self hosted

To run Parmana on your own infrastructure with one command, with the database included, use the self hosted deployment. The checks above still apply.

9. Errors and troubleshooting

Every error body is `{ "error": "<message>", "code": "<CODE>" }`. A few answers carry no `code`; the message then says what is wrong. Every code, with the operations that return it, is in the error reference.

The one rule

Know whether the action may have run before you retry. Parmana's answers fall into three groups:

Group	Answers	Retry?
Refused, nothing ran	`400`, `401`, `403`, `404`, `409` (except `NONCE_ALREADY_CONSUMED`), `429`, `503`	Fix the cause, then send a **new** transaction. `429`: wait for `Retry-After`.
Released, the record is missing	`500 EXECUTION_RECORD_INCOMPLETE`	**No.** The action ran. The operator finalizes the record (Chapter 7).
Released, the outcome unknown	`502 EXECUTION_OUTCOME_UNKNOWN`	**No.** The action may have run. The operator checks the system and resolves the intent.

The SDKs never retry a `POST` on their own; they retry only `GET` requests, and only when you configure a retry policy.

How the SDKs raise them

HTTP	TypeScript (`@parmana/sdk`)	Python (`parmana.errors`)
403 with `POLICY_DENIED`	`ExecutionRejectedError`	`ExecutionRejectedError`
403 with `CAPABILITY_NOT_ALLOWED`	`AuthorizationError` (`serverCode` set)	`AuthorizationError`
400	`ValidationError`	`ValidationError`
401	`AuthenticationError`	`AuthenticationError`
403, any other	`AuthorizationError`	`AuthorizationError`
404	`NotFoundError`	`NotFoundError`
409	`ConflictError`	`ConflictError`
429	`RateLimitError` (`retryAfterSeconds`)	`RateLimitError` (`retry_after_seconds`)
5xx	`InternalServerError`, the code in `serverCode`	`InternalServerError`, the code in `server_code`

Every error from a response carries its status: `statusCode` in TypeScript, `status_code` in Python. Decide on a 5xx by its code, never by the status alone.

By status

400: the request is malformed

Code	Meaning and fix
`AUTHORITY_REQUIRED`, `AUTHORIZATION_REQUIRED`, `INTENT_REQUIRED`	A section of the Business Transaction is missing. Build it with `createBusinessTransaction`.
No code, `approves without a signed human approval`	The policy named cannot load (Chapter 4).
`INVALID_EXTERNAL_CONNECTOR`, `EXTERNAL_ENDPOINT_ADDRESS_REFUSED`	A registration was refused (Chapter 6).
`EXECUTION_INTENT_RESOLUTION_INVALID`	`resolution` must be `NOT_EXECUTED` or `EXECUTED`, with a `note`.

401: no valid key

`{"error":"authentication required"}`, with a `WWW-Authenticate` header. The key is missing, wrong, rotated, or not sent as `Authorization: Bearer <key>`. Nothing about the request was read.

403: not allowed

Code	Meaning and fix
`POLICY_DENIED`	**The policy refused.** A correct answer, with the policy's reason. If the reason asks for an approval, get one signed and send a new transaction (Chapter 5). A Refusal Record was stored.
`CAPABILITY_NOT_ALLOWED`	The key may not use this action. The operator adds it to `allowedCapabilities`.
No code, about the principal	The key may not act for this `principalId`. Act as your own `callerId`.
`NON_HUMAN_CALLER_DENIED`	A governance call from a key not registered as a human.
`SAME_ACTOR_CANNOT_APPROVE_OWN_CHANGE`	The checker is the maker. A different person approves.
`STEP_UP_AUTHORIZATION_INVALID`	The step up signature expired (120 seconds), was used, is for another change or decision, or does not match the checker's registered key. Sign again and send at once.

`POLICY_DENIED` covers every refusal by the decision, each with its own message:

Message starts with	Cause and fix
The policy's own reason, for example `Refund rejected. Every refund needs...`	A rule refused. If it asks for an approval, get one signed and send a new transaction.
`Rejected: capability "x" requires policy "p"@"v", but... was declared.`	The request names another policy or an old version. Read `GET /policies/in-effect` and send that.

Message starts with	Cause and fix
`Rejected: declared signal(s) do not match independently verified state (managerApproved=true != verified managerApproved=false).`	The approval is missing, expired, used, by an untrusted key, for another resource, or below the amount. Sign a new one.
`Rejected: declared signal(s) do not match the executed intent (refundAmount=900 != intent.parameters.amount=750).`	A bound signal does not equal its request field. Copy the value from the request.
`Rejected: no version of policy "p"... has been approved through policy governance`	No approved version yet. Approve one (Chapter 4).

404: not found

Code	Meaning and fix
`CAPABILITY_NOT_BOUND`	No policy is bound to the action: not a built in capability, and no active external registration.
`POLICY_NOT_FOUND`	The policy named does not exist on the server.
`BUSINESS_TRANSACTION_NOT_FOUND`, `EXECUTION_INTENT_NOT_FOUND`	No record with that `businessTransactionId`.
`PENDING_POLICY_CHANGE_NOT_FOUND`, `APPROVAL_ISSUER_CHANGE_NOT_FOUND`, `EXTERNAL_CONNECTOR_CHANGE_NOT_FOUND`	No change with that id.
`VERIFICATION_FAILED`	The record for that id could not be verified or found; the message says which.

409: conflict

Code	Meaning and fix
`DUPLICATE_BUSINESS_TRANSACTION`	This `businessTransactionId` was already used. To retry the same attempt, read its record; for a new attempt, build a new transaction.
`NONCE_ALREADY_CONSUMED`	This exact authorization was already released: the action ran. Read its record; do not send it again as a new transaction.
`NO_APPROVED_POLICY_VERSION`	No version of the bound policy is approved. Every request is refused until one is.
`CONFLICT`	A change is already open, or already resolved, or a registration is already active.
`EXECUTION_INTENT_NOT_RESOLVABLE`	The intent is `RELEASED` (finalize it) or already final.
`EXECUTION_INTENT_RESULT_NOT_RECORDED`	Finalize has nothing to build from: the intent is not `RELEASED`.

429: too many requests

`RATE_LIMITED`. Wait for the `Retry-After` header, then retry the same request.

500, 502: the action may have run

Code	Meaning	Do
`EXECUTION_RECORD_INCOMPLETE`	The action was released; the signed record was not stored.	Do not resend. `POST /execution-intents/{id}/finalize` rebuilds the record without acting again.
`EXECUTION_OUTCOME_UNKNOWN`	The release failed after it may have reached the system: a timeout, an error, an answer that failed the checks.	**Never resend as a new transaction.** Check the system with the `businessTransactionId` and `target`, then `POST /execution-intents/{id}/resolve` with what you found.

For an external connector, a request naming a parameter the registration does not allow, or an endpoint whose host now resolves to a non public address, is also reported as `502 EXECUTION_OUTCOME_UNKNOWN` although nothing was sent. Check the endpoint's own record before resolving.

503: unavailable, nothing ran

Code	Meaning
`CONNECTOR_NOT_REGISTERED`	No connector for the action: not configured, or the external registration is not active.
`SIGNING_UNAVAILABLE`	The signing path is unhealthy. Nothing is released until it is fixed.
`EXECUTION_INTENT_UNAVAILABLE`	The intent store is unavailable (or its table is missing after an upgrade).
`POLICY_VERSION_UNAVAILABLE`	The server could not read the policy in effect.
`AUDIT_UNAVAILABLE`	The audit store is unavailable.

Retry later with a **new** transaction. If it persists, tell the operator; each is an alert in Chapter 8.

Common situations

You see	Cause	Fix
`403 POLICY_DENIED`, `requires policy... but... was declared`	The agent sent an old policy version.	Read `GET /policies/in-effect` before each request; never hard code a version.
Every request refused after a policy change was proposed	The new version is not approved, or the agent names a version not in effect.	Approve it, then read the version in effect.
A server that will not start	A required setting is missing or wrong; the message names the variable.	Environment variable reference.
`PGRST205` "Could not find the table"	The database's schema cache is stale.	`NOTIFY pgrst, 'reload schema';`

More, by symptom: Troubleshooting.

10. Security model and limits

Parmana's guarantees are specific and tested, and each holds within a stated boundary. This chapter states both. The detailed threat model is Limitations; every claim and its evidence is in; every known gap is in, by number.

What Parmana guarantees

For a system whose actions go through Parmana:

Guarantee	Enforced by
Only a key allowed the action can ask for it, and only for the principals it may act for	Caller authentication and the capability check, before any policy runs
Every action is decided by the policy bound to it, at the version two people approved	The capability policy binding and the policy approval record, checked on every request and again at the gateway
No action is approved without a signed approval from a trusted person, for that action and resource	The policy validator (no approve rule without an approval signal) and the approval verifier, before deciding and at release
An approval is used once, within its expiry, for at most its amount	The approval verifier and its nonce store
What is released is exactly what was approved, once, within its lifetime	The signed execution authorization and the gateway's checks: signature, expiry, content hash, single use
The agent never holds a system's credentials	Connectors with per execution credentials; external endpoints hold their own
Every decision, approved or refused, leaves a signed record anyone can verify offline	Trust Records, Refusal Records, Execution Intents, chained audit events
Changes to trust need two human keys	Maker checker with step up signatures for policies, approvers and external connectors
Failures refuse	Every check is fail closed; there is no weaker fallback

What it assumes

- **Actions go through Parmana.** The gateway is a property of systems that route through it. An agent that also holds a system's credentials can act around it; give agents no credentials.
- **Keys stay with their owners.** Parmana proves which key signed, never whose hand held it. Two keys held by one person make the record claim a second person who does not exist.
- **The signing key is safe.** A stolen signing key signs records that verify. AWS KMS keeps the key from being released; rotation is a manual procedure.
- **Policies are written well.** An over broad rule approves what it should not; no signature can detect that. The validator catches some mistakes, and two people review every version.
- **The operator is trusted with the database.** An operator with direct database access can change rows outside maker checker (and, for emergencies, revoke an approver that way); those change rows record it.

What Parmana does not claim

- That execution cannot be bypassed under any circumstances.
- Mathematical proof of correct execution.
- Regulatory compliance (SOC 2, GDPR, ISO 27001) by itself.
- That an external system did what its endpoint says: the answer is recorded as its claim.

Known gaps

The open items that matter most when you build with Parmana. The full list, with the status of each, is under "Open issues at a glance" in.

Gap	What	What it means for you

Also scoped, not absent:

- **There is no automatic path.** No action is authorized without a person, by design. Plan for the first request on every resource to wait for a person.
- **An approval covers the action and the resource**, and the amount where the policy names one, not every parameter.
- **Single use is per nonce store.** Instances that share the database share it; separate in memory stores do not.
- **Route access is not scoped per key.** What is scoped is the action, the principal, and which records a caller can read.
- **Approval notifications are best effort.** One attempt, no retry; the Refusal Records are the complete list.
- **Python verifies Ed25519 only.** A record's ML-DSA-65 signatures are checked by the TypeScript verifier.

Reporting a vulnerability

Email **founder@parmanasystems.com** with the issue, its impact, steps to reproduce and the version affected, as `SECURITY.md` in the repository describes. Do not open a public issue for a vulnerability.

Reference: every endpoint

Every operation the server exposes, 63 in all, read from `openapi/openapi.bundled.yaml`. Tests keep that file equal to the routes the server actually mounts (`openapi-route-parity.test.ts`). Each row links to the operation's page, with its schemas and real captured responses.

Execution

Method	Path	What it does
`POST`	`/execute`	Sends one action for a decision and, if approved, performs it.

Transactions

Method	Path	What it does
`GET`	`/transactions`	Lists the Business Transactions your key submitted, as a bare array with no paging metadata.
`POST`	`/transactions`	The same as `POST /execute` in every way except the success status, `201` instead of `200`.
`GET`	`/transactions/{businessTransactionId}`	Returns one Business Transaction as it was stored.

Verification

Method	Path	What it does
`POST`	`/verify`	Checks a Trust Record again on the server: recomputes its hash, verifies its signature and checks that every approved execution is bound to its authorization.
`GET`	`/verification/{businessTransactionId}`	Returns the most recent Verification appended to a Trust Record by `POST /verify`.

Receipts

Method	Path	What it does
`POST`	`/receipt`	Generates a signed Receipt for a Trust Record: a compact proof of what was decided and executed, to hand to a person or another system.
`GET`	`/receipt/latest/{businessTransactionId}`	Returns the most recent Receipt for a Trust Record.

Trust Records

Method	Path	What it does
`GET`	`/trust-records`	Exports your full, signed Trust Records (transaction, decision, executions, verifications, receipts) a page at a time, for audit and compliance.
`GET`	`/trust-records/{businessTransactionId}`	Returns the complete signed Execution Trust Record for a transaction: what was asked, what the policy decided, what was executed and the connector's answer, with signatures.

Replay

Method	Path	What it does
`POST`	`/replay`	Verifies the stored signature on a Trust Record and returns its hash with the result.

Policies

Method	Path	What it does
`GET`	`/policies/in-effect`	Tells an agent what a request for a capability must declare and carry right now: the policy name and version, the policy's description, and its signals (every fact the rules read with its type, the facts that must equal a value of the in...
`POST`	`/policies/validate`	Checks that this server can load a policy at a name and version, the same lookup a request uses.

Policy Governance

Method	Path	What it does
`POST`	`/policies/{name}/{version}/pending-changes`	Proposes new content for a policy, the maker's step in maker checker governance.
`GET`	`/policies/pending-changes`	Lists policy changes with a diff: the content in effect today (`current` , or `null` for a new version) next to the proposed content.
`POST`	`/policies/pending-changes/{id}/approve`	Approves a policy change, the checker's step, and puts it into effect.
`POST`	`/policies/pending-changes/{id}/reject`	Rejects a policy change.

Approvers

Method	Path	What it does
`GET`	`/approval-issuers`	Lists every approver key the server trusts, or once trusted, to sign human approvals for policies that require one.
`GET`	`/approval-issuers/changes`	Lists approver changes, newest first.

Method	Path	What it does
`POST`	`/approval-issuers/changes`	Proposes trusting a new approver key (`add` , with its Ed25519 public key) or revoking one added this way (`revoke`).
`POST`	`/approval-issuers/changes/{id}/approve`	Approves an approver change and applies it at once: an added key is trusted from the next approval checked; a revoked key's approvals are refused from the next request on, including ones it already signed.
`POST`	`/approval-issuers/changes/{id}/reject`	Rejects an approver change.

External Connectors

Method	Path	What it does
`GET`	`/external-connectors`	Lists every external connector registration, active and revoked: the capability, the HTTPS endpoint Parmana releases it to, the policy that governs it and the parameters forwarded.
`GET`	`/external-connectors/changes`	Lists external connector changes, newest first.
`POST`	`/external-connectors/changes`	Proposes connecting your own system to Parmana with no Parmana code change: `register` names a capability, the HTTPS endpoint to release it to, the policy that governs it, the parameters to forward and optionally a timeout; `revoke` ends...
`POST`	`/external-connectors/changes/{id}/approve`	Approves an external connector change and applies it at once: `register` makes the registration active, `revoke` ends it.
`POST`	`/external-connectors/changes/{id}/reject`	Rejects an external connector change.

Business Signal Sources

Method	Path	What it does
`GET`	`/business-signal-sources`	Lists every business signal source registration, active and revoked: the name a policy's `signalSources` uses, the HTTPS endpoint Parmana asks, and the Ed25519 key its answers must be signed with, if one is registered.
`GET`	`/business-signal-sources/changes`	Lists business signal source changes, newest first.
`POST`	`/business-signal-sources/changes`	Proposes connecting a business system, any system that owns facts a policy needs (an ERP, an order system, a CRM, a ledger), so Parmana can ask it instead of trusting the agent's claim.
`POST`	`/business-signal-sources/changes/{id}/approve`	Approves a business signal source change and applies it at once: `register` makes the registration active, so policies that name it can be evaluated; `revoke` ends it, so they are refused as `SOURCE_UNAVAILABLE` .

Method	Path	What it does
`POST`	`/business-signal-sources/changes/{id}/reject`	Rejects a business signal source change.

Authority Grants

Method	Path	What it does
`GET`	`/authority-grants`	Lists every authority grant, active and revoked: which caller may have which capability decided, within which limits, from when until when.
`GET`	`/authority-grants/changes`	Lists authority grant changes, newest first.
`POST`	`/authority-grants/changes`	Proposes per-agent authority (RFC-0023 phase 3): `grant` says the API key with `callerId` may have requests for `capability` decided, within `limits` on the request, until `validUntil` (at most 366 days); `revoke` ends the active grant.
`POST`	`/authority-grants/changes/{id}/approve`	Approves an authority grant change and applies it at once: `grant` makes the grant active, starting now unless it names a later `validFrom`; `revoke` ends it, so the caller's next request under a policy with `requireAuthorityGrant` is re...
`POST`	`/authority-grants/changes/{id}/reject`	Rejects an authority grant change.

Sandbox

Method	Path	What it does
`POST`	`/sandbox/approvals`	**Sandbox only.** Returns an approval signed by the sandbox's demo approver, so you can try an approved request from a browser.

Kill Switch

Method	Path	What it does
`POST`	`/kill-switch/trip`	Stops new authorization and approval requests until an operator resets it.
`POST`	`/kill-switch/reset`	Lets authorization and approval requests through again.
`GET`	`/kill-switch/status`	Returns the current state with the nonce you sent echoed back.
`POST`	`/kill-switch/probes`	Records what a scheduler saw from a harmless authorization request.

Refusal Records

Method	Path	What it does
`POST`	`/refusal/verify`	Checks a Refusal Record's signature.

Method	Path	What it does
`GET`	`/refusal/{businessTransactionId}`	Returns the signed Refusal Record kept when a policy refused a transaction: the decision, the rule that refused, the facts it saw and the signature.

Execution Intents

Method	Path	What it does
`POST`	`/execution-intents/verify`	Checks an Execution Intent's hash and signature.
`GET`	`/execution-intents/unfinalized`	The operator's work queue: Execution Intents that never reached a signed Trust Record and were not closed by hand, oldest first.
`GET`	`/execution-intents/{businessTransactionId}`	Returns the signed Execution Intent for a transaction and where it got to.
`POST`	`/execution-intents/{businessTransactionId}/finalize`	Rebuilds the signed Trust Record for an action that was released but whose record was never produced, the repair for a `500` `EXECUTION_RECORD_INCOMPLETE`.
`POST`	`/execution-intents/{businessTransactionId}/resolve`	Closes an intent in state `PREPARED` or `ERRORED` after a person checked the target system, the repair for a `502` `EXECUTION_OUTCOME_UNKNOWN`.

Audit

Method	Path	What it does
`POST`	`/audit/verify`	Checks the signature of a caller authentication audit event, the record the server keeps of every authentication, denial and governance action.

System

Method	Path	What it does
`GET`	`/`	Returns the service name and `UP`.
`GET`	`/health`	Liveness probe.
`GET`	`/ready`	Readiness probe.
`GET`	`/openapi.yaml`	This API's complete OpenAPI 3.1 document as YAML, self contained with no external references.
`GET`	`/openapi.json`	The same document as `GET /openapi.yaml`, as JSON, for tools and agents that expect a spec at a `.json` path.
`GET`	`/api-manifest.json`	One index for agents and tools: where the OpenAPI document and the rendered references are, how to authenticate, and both official SDKs with their package name, registry and current version.
`GET`	`/version`	Returns `name`, `version` and `api`.

Method	Path	What it does
`GET`	`/keys/{keyId}`	Returns the public half of a signing key, so anyone can verify a Parmana signature without a Parmana account: "an auditor, a regulator, or an endpoint receiving a release.
`GET`	`/.well-known/jwks.json`	Lists every public signing key this deployment can serve, the enumeration form of `GET /keys/{keyId}`.
`GET`	`/callers/me`	Returns who your key is and exactly what it may do: the caller id, the principals it may act for, the capabilities it may invoke and whether that includes every capability (`*`), after defaults are applied.

Handbook

Method	Path	What it does
`GET`	`/parmana-handbook.pdf`	Downloads the Parmana Handbook as a PDF.
`GET`	`/handbook/download-leads`	Records an email address, then redirects to the handbook PDF.
`POST`	`/handbook/download-leads`	Records an email address and returns the handbook download link as JSON.

Reference: every SDK method

Read from the SDK source with each method's own doc comment: `typescript/src/client/ParmanaClient.ts` and `python/parmana/client.py`. A method added to either client appears here the next time the reference is generated, and a test fails until it is. Configuration, retries and errors: TypeScript and Python.

TypeScript: `ParmanaClient` (`@parmana/sdk`)

Signature	What it does
`toJSON(): { endpoint: string }`	
`endpoint(): string`	Returns the configured Runtime endpoint.
`health(): Promise<HealthStatus>`	Performs a Runtime health check.
`version(): Promise<unknown>`	Returns the Runtime name, version, and API version.
`execute(transaction: BusinessTransaction): Promise<ExecutionTrustRecord>`	Executes a Business Transaction.
`verify(businessTransactionId: string): Promise<Verification>`	Runs a fresh verification of an Execution Trust Record, appending a new Verification to its history. Distinct from getLatestVerification(), which reads the most recent one without re-verifying.
`getLatestVerification(businessTransactionId: string): Promise<Verification>`	Returns the latest Verification.
`replay(businessTransactionId: string): Promise<ReplayResult>`	Performs deterministic replay.
`receipt(businessTransactionId: string): Promise<Receipt>`	Generates an execution receipt.
`createTransaction(transaction: BusinessTransaction): Promise<ExecutionTrustRecord>`	Creates (executes) a Business Transaction via POST /transactions, a second, independent entry point into the identical execution pipeline as execute() (POST /execute). See TransactionApi.create.
`transaction(businessTransactionId: string): Promise<BusinessTransaction>`	Retrieves a Business Transaction.
`transactions(page = 1, pageSize = 25): Promise<BusinessTransaction[]>`	Lists Business Transactions.
`latestReceipt(businessTransactionId: string): Promise<Receipt>`	Retrieves the most recent receipt without generating a new one.
`trustRecord(businessTransactionId: string): Promise<ExecutionTrustRecord>`	Retrieves an Execution Trust Record.
`trustRecords(page = 1, pageSize = 25, options: { readonly since?: string; readonly until?: string } = {}): Promise<ExecutionTrustRecord[]>`	Lists Execution Trust Records of this caller's transactions, newest first. See TrustRecordApi.list.

Signature	What it does
`caller(): Promise<CallerIdentity>`	Who this API key belongs to and what it may do (GET /callers/me).
`publicKey(keyId = "default"): Promise<PublicKeyInfo>`	A signing public key of the deployment (GET /keys/{keyId}), for the offline verifiers. Records are signed with `default`.
`policyInEffect(capability: string): Promise<PolicyInEffect>`	The policy a request for `capability` must declare right now, and what the request must carry (GET /policies/in-effect). Call it before every request and declare `policy` exactly as returned.
`validatePolicy(policyId: string, policyVersion: string): Promise<PolicyValidationResult>`	Confirms that a policy (name + version) is loadable by the Runtime.
`refusalRecord(businessTransactionId: string): Promise<RefusalRecord>`	Retrieves a Refusal Record by Business Transaction ID.
`verifyRefusalRecord(record: RefusalRecord): Promise<boolean>`	Verifies a Refusal Record's signature.
`executionIntent(businessTransactionId: string): Promise<ExecutionIntentView>`	Retrieves an Execution Intent and its status by Business Transaction ID (ADR-0012). The intent is the signed statement, stored before an action is released, of what was about to be released.
`verifyExecutionIntent(intent: ExecutionIntent): Promise<boolean>`	Verifies an Execution Intent's hash and signature. Needs no credential. `true` proves the intent is genuine and unaltered. It does not prove the action was released, or what its result was.
`unfinalizedExecutionIntents(limit?: number): Promise<UnfinalizedExecutionIntents>`	Lists Execution Intents that never reached a signed Trust Record and were not closed by hand, oldest first. Needs a credential provisioned as a verified human.
`finalizeExecutionIntent(businessTransactionId: string): Promise<FinalizeExecutionIntentResult>`	Rebuilds the signed Trust Record for a released action whose record was never produced. Never calls a connector. Safe to run twice. Needs a credential provisioned as a verified human.
`resolveExecutionIntent(businessTransactionId: string, input: ResolveExecutionIntentInput): Promise<ResolveExecutionIntentResult>`	Closes a PREPARED or ERRORED intent that a verified human reconciled at the connector. The note is required. The resolution is an attributed operator statement in unsigned status, not a Trust Record.
`verifyAuditEvent(event: AuditEvent, signature: Signature): Promise<boolean>`	Verifies a signed caller-authentication audit event's signature.
`proposePolicyChange(name: string, version: string, input: ProposePolicyChangeInput): Promise<ProposedPolicyChange>`	Proposes a policy change. See PolicyApi.proposeChange.
`policyChanges(status?: PendingPolicyChangeStatus): Promise<PolicyChangeForReview[]>`	Lists policy changes for review. See PolicyApi.listChanges.
`approvePolicyChange(pendingPolicyChangeId: string, stepUpAuthorization: PolicyChangeStepUpAuthorization): Promise<PendingPolicyChange>`	Approves a policy change with a signed step up authorization. See PolicyApi.approveChange and signPolicyChangeStepUp().

Signature	What it does
`rejectPolicyChange(pendingPolicyChangeId: string, rejectionReason: string, stepUpAuthorization: PolicyChangeStepUpAuthorization): Promise<PendingPolicyChange>`	Rejects a policy change with a reason and a signed step up authorization. See PolicyApi.rejectChange.
`approvers(): Promise<ApprovalIssuer[]>`	Every approver key the server trusts or trusted. See ApproverApi.list.
`proposeApproverChange(input: ProposeApproverChangeInput): Promise<ApprovalIssuerChange>`	Proposes adding or revoking an approver key. See ApproverApi.proposeChange.
`approverChanges(status?: PendingPolicyChangeStatus): Promise<ApprovalIssuerChange[]>`	Lists approver changes. See ApproverApi.listChanges.
`approveApproverChange(changeId: string, stepUpAuthorization: PolicyChangeStepUpAuthorization): Promise<ApprovalIssuerChange>`	Approves and applies an approver change. See ApproverApi.approveChange.
`rejectApproverChange(changeId: string, rejectionReason: string, stepUpAuthorization: PolicyChangeStepUpAuthorization): Promise<ApprovalIssuerChange>`	Rejects an approver change. See ApproverApi.rejectChange.
`externalConnectors(): Promise<ExternalConnector[]>`	Every external connector registration, active and revoked. See ExternalConnectorApi.list.
`proposeExternalConnectorChange(input: ProposeExternalConnectorChangeInput): Promise<ExternalConnectorChange>`	Proposes registering or revoking an external connector. See ExternalConnectorApi.proposeChange.
`externalConnectorChanges(status?: PendingPolicyChangeStatus): Promise<ExternalConnectorChange[]>`	Lists external connector changes. See ExternalConnectorApi.listChanges.
`approveExternalConnectorChange(changeId: string, stepUpAuthorization: PolicyChangeStepUpAuthorization): Promise<ExternalConnectorChange>`	Approves and applies an external connector change. See ExternalConnectorApi.approveChange.
`rejectExternalConnectorChange(changeId: string, rejectionReason: string, stepUpAuthorization: PolicyChangeStepUpAuthorization): Promise<ExternalConnectorChange>`	Rejects an external connector change. See ExternalConnectorApi.rejectChange.
`businessSignalSources(): Promise<BusinessSignalSource[]>`	Every business signal source registration, active and revoked. See BusinessSignalSourceApi.list.
`proposeBusinessSignalSourceChange(input: ProposeBusinessSignalSourceChangeInput): Promise<BusinessSignalSourceChange>`	Proposes registering or revoking a business signal source. See BusinessSignalSourceApi.proposeChange.

Signature	What it does
`businessSignalSourceChanges(status?: PendingPolicyChangeStatus): Promise<BusinessSignalSourceChange[]>`	Lists business signal source changes. See BusinessSignalSourceApi.listChanges.
`approveBusinessSignalSourceChange(changeId: string, stepUpAuthorization: PolicyChangeStepUpAuthorization): Promise<BusinessSignalSourceChange>`	Approves and applies a business signal source change. See BusinessSignalSourceApi.approveChange.
`rejectBusinessSignalSourceChange(changeId: string, rejectionReason: string, stepUpAuthorization: PolicyChangeStepUpAuthorization): Promise<BusinessSignalSourceChange>`	Rejects a business signal source change. See BusinessSignalSourceApi.rejectChange.
`authorityGrants(): Promise<AuthorityGrant[]>`	Every authority grant, active and revoked. See AuthorityGrantApi.list.
`proposeAuthorityGrantChange(input: ProposeAuthorityGrantChangeInput): Promise<AuthorityGrantChange>`	Proposes granting or revoking an agent's authority. See AuthorityGrantApi.proposeChange.
`authorityGrantChanges(status?: PendingPolicyChangeStatus): Promise<AuthorityGrantChange[]>`	Lists authority grant changes. See AuthorityGrantApi.listChanges.
`approveAuthorityGrantChange(changeId: string, stepUpAuthorization: PolicyChangeStepUpAuthorization): Promise<AuthorityGrantChange>`	Approves and applies an authority grant change. See AuthorityGrantApi.approveChange.
`rejectAuthorityGrantChange(changeId: string, rejectionReason: string, stepUpAuthorization: PolicyChangeStepUpAuthorization): Promise<AuthorityGrantChange>`	Rejects an authority grant change. See AuthorityGrantApi.rejectChange.

TypeScript: functions exported next to the client

They make no request.

Signature	What it does
`verifyExecutionTrustRecordOffline(trustRecord: unknown, publicKeys: PublicKeys): OfflineVerificationResult`	Verifies an Execution Trust Record with only public keys.
`verifyExecutionIntentOffline(intent: unknown, publicKeys: PublicKeys): OfflineVerificationResult`	Verifies an Execution Intent with only public keys.

Signature	What it does
`signPolicyChangeStepUp(input: SignPolicyChangeStepUpInput): PolicyChangeStepUpAuthorization`	Signs a step up authorization for one decision (approve or reject) on one change: a policy, an approver or an external connector. Valid for ttlSeconds (default 120) and usable once. Run by the checker, with their own step up private key.
`signApproval(input: SignApprovalInput): SignedApproval`	Signs an approval for one action on one resource, optionally up to an amount, valid for ttlSeconds (default 900, at most 86400) and usable once. Run by the approver, with their own private key.
`verifyParmanaRelease(body: unknown, options: VerifyParmanaReleaseOptions): Promise<ParmanaReleaseVerification>`	Checks, in order: the body's shape, the signature over the canonical JSON of `release` with the key named in signature.keyId, that release.audience equals `audience`, and that release.expiresAt has not passed (with the clock skew allowance). Only then does it ask isAlreadyExecuted. Never throws for a bad body; returns `{ valid: false, errors }`.

Python: `ParmanaClient` (`parmana`)

Signature	What it does
`endpoint() -> str`	Parmana Runtime endpoint.
`version() -> str`	Parmana SDK version.
`health() -> dict[str, Any]`	Returns the Runtime health status.
`execute(transaction: BusinessTransaction) -> ExecutionTrustRecord`	Execute a Business Transaction.
`verify(business_transaction_id: str) -> Verification`	Run a fresh verification of an Execution Trust Record, appending a new Verification to its history. Distinct from get_latest_verification(), which reads the most recent one without re-verifying.
`get_latest_verification(business_transaction_id: str) -> Verification`	Returns the latest Verification, without performing a fresh one.
`create_transaction(transaction: BusinessTransaction) -> ExecutionTrustRecord`	Creates (executes) a Business Transaction via POST /transactions, a second, independent entry point into the identical execution pipeline as execute() (POST /execute).
`transaction(business_transaction_id: str) -> BusinessTransaction`	Retrieves a Business Transaction.
`trust_record(business_transaction_id: str) -> ExecutionTrustRecord`	Retrieves an Execution Trust Record.
`policy_in_effect(capability: str) -> PolicyInEffect`	The policy a request for `capability` must declare right now, and what the request must carry (GET /policies/in-effect). Call it before every request and declare `policy` exactly as returned.
`validate_policy(policy_id: str, policy_version: str) -> dict[str, Any]`	Validates that a policy (name + version) is loadable.

Signature	What it does
`refusal_record(business_transaction_id: str) -> RefusalRecord`	Retrieves a Refusal Record by Business Transaction ID.
`verify_refusal_record(record: RefusalRecord) -> bool`	Verifies a Refusal Record's signature.
`execution_intent(business_transaction_id: str) -> ExecutionIntentView`	Retrieves an Execution Intent and its status (ADR-0012). The intent is the signed statement, stored before an action is released, of what was about to be released.
`verify_execution_intent(intent: ExecutionIntent) -> bool`	Verifies an Execution Intent's hash and signature. Needs no credential. True proves the intent is genuine and unaltered. It does not prove the action was released, or what its result was.
`unfinalized_execution_intents(limit: int | None = None) -> UnfinalizedExecutionIntents`	Lists Execution Intents that never reached a signed Trust Record and were not closed by hand, oldest first. Needs a credential provisioned as a verified human.
`finalize_execution_intent(business_transaction_id: str) -> FinalizeExecutionIntentResult`	Rebuilds the signed Trust Record for a released action whose record was never produced. Never calls a connector. Safe to run twice. Needs a credential provisioned as a verified human.
`resolve_execution_intent(business_transaction_id: str, *, resolution: ExecutionIntentResolution | str, note: str) -> ResolveExecutionIntentResult`	Closes a PREPARED or ERRORED intent that a verified human reconciled at the connector. The note is required. The resolution is an attributed operator statement in unsigned status, not a Trust Record.
`verify_audit_event(event: dict[str, Any], signature: Signature) -> bool`	Verifies a signed caller-authentication or Razorpay-webhook audit event's signature.
`latest_receipt(business_transaction_id: str) -> Receipt`	Retrieve the most recent receipt without generating a new one. See ReceiptApi.get_latest.
`caller() -> CallerIdentity`	Who this API key belongs to and what it may do (GET /callers/me).
`public_key(key_id: str = "default") -> PublicKeyInfo`	A signing public key of the deployment (GET /keys/{keyId}), for the offline verifiers. Records are signed with "default".
`propose_policy_change(name: str, version: str, *, proposed_content: dict[str, Any], reason: str) -> ProposedPolicyChange`	Propose a policy change. See PolicyApi.propose_change.
`policy_changes(status: str | None = None) -> list[PolicyChangeForReview]`	List policy changes for review. See PolicyApi.list_changes.
`approve_policy_change(pending_policy_change_id: str, step_up_authorization: dict[str, Any]) -> PendingPolicyChange`	Approve a policy change with a signed step up authorization. See PolicyApi.approve_change and parmana.crypto.sign_policy_change_step_up.
`reject_policy_change(pending_policy_change_id: str, rejection_reason: str, step_up_authorization: dict[str, Any]) -> PendingPolicyChange`	Reject a policy change with a reason and a signed step up authorization. See PolicyApi.reject_change.
`list_approvers() -> list[ApprovalIssuer]`	Every approver key the server trusts or trusted. See ApproverApi.list.

Signature	What it does
`approver_changes(status: str | None = None) -> list[ApprovalIssuerChange]`	List approver changes. See ApproverApi.list_changes.
`approve_approver_change(change_id: str, step_up_authorization: dict[str, Any]) -> ApprovalIssuerChange`	Approve and apply an approver change. See ApproverApi.approve_change.
`reject_approver_change(change_id: str, rejection_reason: str, step_up_authorization: dict[str, Any]) -> ApprovalIssuerChange`	Reject an approver change. See ApproverApi.reject_change.
`list_external_connectors() -> list[ExternalConnector]`	Every external connector registration, active and revoked. See ExternalConnectorApi.list.
`external_connector_changes(status: str | None = None) -> list[ExternalConnectorChange]`	List external connector changes. See ExternalConnectorApi.list_changes.
`approve_external_connector_change(change_id: str, step_up_authorization: dict[str, Any]) -> ExternalConnectorChange`	Approve and apply an external connector change. See ExternalConnectorApi.approve_change.
`reject_external_connector_change(change_id: str, rejection_reason: str, step_up_authorization: dict[str, Any]) -> ExternalConnectorChange`	Reject an external connector change. See ExternalConnectorApi.reject_change.
`list_business_signal_sources() -> list[BusinessSignalSource]`	Every business signal source registration, active and revoked. See BusinessSignalSourceApi.list.
`business_signal_source_changes(status: str | None = None) -> list[BusinessSignalSourceChange]`	List business signal source changes. See BusinessSignalSourceApi.list_changes.
`approve_business_signal_source_change(change_id: str, step_up_authorization: dict[str, Any]) -> BusinessSignalSourceChange`	Approve and apply a business signal source change. See BusinessSignalSourceApi.approve_change.
`reject_business_signal_source_change(change_id: str, rejection_reason: str, step_up_authorization: dict[str, Any]) -> BusinessSignalSourceChange`	Reject a business signal source change. See BusinessSignalSourceApi.reject_change.
`list_authority_grants() -> list[AuthorityGrant]`	Every authority grant, active and revoked. See AuthorityGrantApi.list.
`authority_grant_changes(status: str | None = None) -> list[AuthorityGrantChange]`	List authority grant changes. See AuthorityGrantApi.list_changes.

Signature	What it does
`approve_authority_grant_change(change_id: str, step_up_authorization: dict[str, Any]) -> AuthorityGrantChange`	Approve and apply an authority grant change. See AuthorityGrantApi.approve_change.
`reject_authority_grant_change(change_id: str, rejection_reason: str, step_up_authorization: dict[str, Any]) -> AuthorityGrantChange`	Reject an authority grant change. See AuthorityGrantApi.reject_change.

Python: functions in `parmana.crypto`

They make no request. Install with `pip install "parmana[verify]"`.

Signature	What it does
`verify_execution_trust_record_offline(trust_record: Any, public_keys: dict[str, str]) -> OfflineVerificationResult`	`trust_record` is a plain dict, exactly as `json.load()` would produce from a Trust Record JSON file or API response body, or the ExecutionTrustRecord that `client.trust_record()` returns. `public_keys` maps keyId → PEM-encoded public key text.
`verify_execution_intent_offline(intent: Any, public_keys: dict[str, str]) -> OfflineVerificationResult`	Python counterpart to verifyExecutionIntentOffline() in packages/crypto/src/OfflineVerifier.ts (ADR-0012). No network call, no database, no environment variable: only the intent and the public key(s).
`sign_policy_change_step_up(*, pending_policy_change_id: str, action: str, private_key_pem: str, key_id: str, ttl_seconds: float = DEFAULT_TTL_SECONDS) -> dict[str, Any]`	Sign a step up authorization for one action on one policy change.
`sign_approval(*, private_key_pem: str, approver_id: str, key_id: str, capability: str, resource_id: str, max_amount: float | None = None, ttl_seconds: int = DEFAULT_APPROVAL_TTL_SECONDS) -> dict[str, Any]`	Sign an approval of one action on one resource.
`verify_parmana_release(body: Any, *, public_keys: dict[str, str], audience: str, is_already_executed: Callable[[str], bool], now: datetime | None = None, clock_skew_seconds: float = DEFAULT_RELEASE_CLOCK_SKEW_SECONDS) -> ParmanaReleaseVerification`	Checks, in order: the body's shape, the signature over the canonical JSON of `release` with the key named in `signature.keyId`, that `release.audience` equals `audience` (this endpoint's URL exactly as Parmana stored it at registration), and that `release.expiresAt` has not passed, allowing `clock_skew_seconds`. Only then does it call `is_already_executed(businessTransactionId)`.
`canonical_serialize(value: Any) -> bytes`	Serializes an already-JSON-decoded value into the same canonical UTF-8 bytes CanonicalSerializer.ts would produce for the equivalent JavaScript value.

Reference: every error code

Every error code the server declares, 46 in all: each error class in `packages/*/src` (its code and HTTP status), and each `code` in a real captured response in `openapi/openapi.bundled.yaml`. An error body is `{ "error": "<message>", "code": "<CODE>" }`; a few refusals carry more fields, shown on the operation's page. "Returned by" lists the operations whose documented responses name the code; an empty cell means the code can be returned by any operation that reaches that part of the server. What to do for each class of error: Errors and troubleshooting.

Code	HTTP	Returned by	Example message	Declared in
`APPROVAL_ISSUER_CHANGE_NOT_FOUND`	404	`approveApprovalIssuerChange`, `rejectApprovalIssuerChange`	Approver change '00000000-0000-0000-0000-000000000000' not found.	`packages/shared/src/errors/approval-issuer-change-not-found-error.ts`
`AUDIT_UNAVAILABLE`	503			`packages/api/src/auth/AuditUnavailableError.ts`
`AUTHORITY_GRANT_CHANGE_NOT_FOUND`	404	`approveAuthorityGrantChange`, `rejectAuthorityGrantChange`	Authority grant change '00000000-0000-0000-0000-000000000000' not found.	`packages/shared/src/errors/authority-grant-change-not-found-error.ts`
`AUTHORITY_REQUIRED`	400			`packages/shared/src/errors/authority-required-error.ts`
`AUTHORITY_SELF_GRANT_DENIED`	403	`approveAuthorityGrantChange`, `proposeAuthorityGrantChange`, `rejectAuthorityGrantChange`	'grant-maker' may not propose, approve or reject authority for itself.	`packages/api/src/routes/authority-grants.ts`

Code	HTTP	Returned by	Example message	Declared in
`AUTHORIZATION_NO_LONGER_VALID`	409	`executeTransaction`	Not released: what the authorization for business transaction '38dcf000-99ed-43af-8666-46c63c8ada74' rests on no longer holds (authority grant 9f3cf733-a8ed-487c-bb58-a53950388cdf is no longer the active grant of "warehouse-agent" for "erp:release-goods" (revoked or replaced)). Nothing was executed. Send a new request to be decided again.	`packages/shared/src/errors/authorization-no-longer-valid-error.ts`
`AUTHORIZATION_REQUIRED`	400			`packages/shared/src/errors/authorization-required-error.ts`
`BUSINESS_SIGNAL_SOURCE_CHANGE_NOT_FOUND`	404	`approveBusinessSignalSourceChange`, `rejectBusinessSignalSourceChange`	Business signal source change '00000000-0000-0000-0000-000000000000' not found.	`packages/shared/src/errors/business-signal-source-change-not-found-error.ts`
`BUSINESS_TRANSACTION_NOT_FOUND`	404			`packages/shared/src/errors/business-transaction-not-found-error.ts`
`CAPABILITY_NOT_ALLOWED`	403	`createTransaction`, `executeTransaction`, `getPolicyInEffect`	Caller is not permitted to invoke this capability.	`packages/api/src/routes/execute.ts`
`CAPABILITY_NOT_BOUND`	404	`getPolicyInEffect`	No policy is bound to capability "test:fixture-execute".	`packages/api/src/routes/policy-in-effect.ts`

Code	HTTP	Returned by	Example message	Declared in
`CHALLENGE_RECORD_NOT_FOUND`	404			`packages/shared/src/errors/challenge-record-not-found-error.ts`
`CONFLICT`	409	`approveApprovalIssuerChange`, `approveAuthorityGrantChange`, `approveBusinessSignalSourceChange`, `approveExternalConnectorChange`, `proposeApprovalIssuerChange`, `proposeAuthorityGrantChange`, `proposeBusinessSignalSourceChange`, `proposeExternalConnectorChange`, `rejectApprovalIssuerChange`, `rejectAuthorityGrantChange`, `rejectBusinessSignalSourceChange`, `rejectExternalConnectorChange`	Approver 'approver-example' key 'approver-example-key-1' already exists. Use a new key id to add a new key.	
`CONNECTOR_NOT_REGISTERED`	503			`packages/shared/src/errors/connector-not-registered-error.ts`
`DECISION_NOT_APPROVED`	400			`packages/shared/src/errors/decision-not-approved-error.ts`
`DUPLICATE_BUSINESS_TRANSACTION`	409			`packages/shared/src/errors/duplicate-business-transaction-error.ts`
`EXECUTION_INTENT_NOT_FOUND`	404	`finalizeExecutionIntent`, `resolveExecutionIntent`	No Execution Intent exists for business transaction 'unknown-transaction'. Transactions created before Execution Intents were introduced have none.	`packages/runtime/src/errors/ExecutionIntentNotFoundError.ts`

Code	HTTP	Returned by	Example message	Declared in
`EXECUTION_INTENT_NOT_RESOLVABLE`	409	`resolveExecutionIntent`	The Execution Intent for business transaction 'b1237ce3-305a-4004-8179-749352fc1ade' is in state RELEASED and cannot be resolved by hand. Its execution result was saved, so rebuild the signed Trust Record with finalize instead. Nothing was changed.	`packages/runtime/src/errors/ExecutionIntentNotResolvableError.ts`
`EXECUTION_INTENT_RESOLUTION_INVALID`	400	`resolveExecutionIntent`	The resolution is invalid: note is required. Record what you found at the connector.	`packages/runtime/src/errors/ExecutionIntentResolutionInvalidError.ts`
`EXECUTION_INTENT_RESULT_NOT_RECORDED`	409	`finalizeExecutionIntent`	The Execution Intent for business transaction 'b1237ce3-305a-4004-8179-749352fc1ade' is in state PREPARED and has no saved execution result, so its signed Execution Trust Record cannot be rebuilt. The action may or may not have been released. Establish the outcome from the connector and reconcile it by hand. Nothing was called.	`packages/runtime/src/errors/ExecutionIntentNotFinalizableError.ts`
`EXECUTION_INTENT_UNAVAILABLE`	503			`packages/runtime/src/errors/ExecutionIntentUnavailableError.ts`
`EXECUTION_INTENTS_NOT_ENABLED`	501	`finalizeExecutionIntent`, `resolveExecutionIntent`	Execution Intents are not enabled on this deployment.	
`EXECUTION_OUTCOME_UNKNOWN`	502	`createTransaction`, `executeTransaction`	The action for business transaction '38dcf000-99ed-43af-8666-46c63c8ada74' was released to the execution system, but the call failed, so whether it was performed is unknown. Do not retry as a new transaction: check the target system, then close the Execution Intent with POST /execution-intents/38dcf000-99ed-43af-8666-46c63c8ada74/resolve. authorizationId: 4d291c90-9e93-4a8a-b214-8cc6526fe2e3.	`packages/runtime/src/errors/ExecutionOutcomeUnknownError.ts`

Code	HTTP	Returned by	Example message	Declared in
`EXECUTION_RECORD_INCOMPLETE`	500	`createTransaction`, `executeTransaction`	The action for business transaction 'e54187e6-431f-4e1c-9c0e-ff2cb5d6d40a' was released to the execution system, but its signed Execution Trust Record could not be produced. Do not retry as a new transaction: reconcile against the connector and the execution audit events for this businessTransactionId. authorizationId: 381ef27a-0955-4085-9e5d-e4223fef2986.	`packages/runtime/src/errors/ExecutionRecordIncompleteError.ts`
`EXTERNAL_CONNECTOR_CHANGE_NOT_FOUND`	404	`approveExternalConnectorChange`, `rejectExternalConnectorChange`	External connector change '00000000-0000-0000-0000-000000000000' not found.	`packages/shared/src/errors/external-connector-change-not-found-error.ts`
`EXTERNAL_ENDPOINT_ADDRESS_REFUSED`	400	`approveBusinessSignalSourceChange`, `approveExternalConnectorChange`, `proposeBusinessSignalSourceChange`, `proposeExternalConnectorChange`	The endpoint host 'erp-internal.example.com' resolves to an address that is not public (10.0.0.7).	`packages/api/src/routes/business-signal-sources.ts`
`INTENT_REQUIRED`	400			`packages/shared/src/errors/intent-required-error.ts`
`INVALID_AUTHORITY_GRANT`	400	`approveAuthorityGrantChange`, `proposeAuthorityGrantChange`	capability must be namespace:verb, at most 128 characters, matching ^[a-z][a-z0-9](?:[-_][a-z0-9]+):[a-z][a-z0-9](?:[-_][a-z0-9]+)$.	`packages/api/src/routes/authority-grants.ts`
`INVALID_BUSINESS_SIGNAL_SOURCE`	400	`proposeBusinessSignalSourceChange`	name must match ^[a-z0-9][a-z0-9-]{0,62}$, the form a policy's signalSources uses.	`packages/api/src/routes/business-signal-sources.ts`
`INVALID_EXTERNAL_CONNECTOR`	400	`proposeExternalConnectorChange`	The namespace 'paytm' belongs to a built in connector. Use another namespace.	`packages/api/src/routes/external-connectors.ts`

Code	HTTP	Returned by	Example message	Declared in
`INVALID_SANDBOX_APPROVAL_REQUEST`	400	`createSandboxApproval`	The sandbox approver signs only for capability "sandbox:receipt".	`packages/api/src/routes/sandbox-approvals.ts`
`NO_APPROVED_POLICY_VERSION`	409	`getPolicyInEffect`	Every request for "paytm:refund" is refused right now: no version of policy "customer-refund", which capability "paytm:refund" requires, has been approved through policy governance.	`packages/api/src/routes/policy-in-effect.ts`
`NON_HUMAN_CALLER_DENIED`	403	`approveApprovalIssuerChange`, `approveAuthorityGrantChange`, `approveBusinessSignalSourceChange`, `approveExternalConnectorChange`, `approvePolicyChange`, `finalizeExecutionIntent`, `listApprovalIssuerChanges`, `listApprovalIssuers`, `listAuthorityGrantChanges`, `listAuthorityGrants`, `listBusinessSignalSourceChanges`, `listBusinessSignalSources`, `listExternalConnectorChanges`, `listExternalConnectors`, `listPendingPolicyChanges`, `listUnfinalizedExecutionIntents`, `proposeApprovalIssuerChange`, `proposeAuthorityGrantChange`, `proposeBusinessSignalSourceChange`, `proposeExternalConnectorChange`, `proposePolicyChange`, `rejectApprovalIssuerChange`, `rejectAuthorityGrantChange`, `rejectBusinessSignalSourceChange`, `rejectExternalConnectorChange`, `rejectPolicyChange`, `resolveExecutionIntent`	This action requires a caller credential provisioned as a verified human (credentialHolderType: USER).	`packages/shared/src/errors/non-human-caller-denied-error.ts`
`NONCE_ALREADY_CONSUMED`	409			`packages/shared/src/errors/nonce-already-consumed-error.ts`

Code	HTTP	Returned by	Example message	Declared in
`OVERRIDE_NOT_ALLOWED`	409			`packages/shared/src/errors/override-not-allowed-error.ts`
`PENDING_POLICY_CHANGE_NOT_FOUND`	404	`approvePolicyChange`, `rejectPolicyChange`	Pending Policy Change '00000000-0000-0000-0000-000000000000' not found.	`packages/shared/src/errors/pending-policy-change-not-found-error.ts`
`POLICY_DENIED`	403	`createTransaction`, `executeTransaction`	Execution rejected: Vendor payment rejected because the assessed payment risk exceeds the maximum permitted threshold.	
`POLICY_NOT_FOUND`	404			`packages/shared/src/errors/policy-not-found-error.ts`
`POLICY_VERSION_UNAVAILABLE`	503	`getPolicyInEffect`	Every request for "paytm:refund" is refused right now: the version of policy "customer-refund" in effect for capability "paytm:refund" could not be looked up: database unreachable.	`packages/api/src/routes/policy-in-effect.ts`
`RATE_LIMITED`	429			`packages/api/src/middleware/rate-limit.ts`
`RECEIPT_GENERATION_FAILED`	409	`generateReceipt`	Execution Trust Record must be successfully verified before a Receipt can be generated.	`packages/runtime/src/errors/ReceiptGenerationError.ts`

Code	HTTP	Returned by	Example message	Declared in
`SAME_ACTOR_CANNOT_APPROVE_OWN_CHANGE`	403	`approveApprovalIssuerChange`, `approveAuthorityGrantChange`, `approveBusinessSignalSourceChange`, `approveExternalConnectorChange`, `approvePolicyChange`, `rejectApprovalIssuerChange`, `rejectAuthorityGrantChange`, `rejectBusinessSignalSourceChange`, `rejectExternalConnectorChange`, `rejectPolicyChange`	Pending Policy Change '307d8265-1171-49a4-b13f-fc3d3ae323e3' was proposed by this same caller — the proposer (maker) may not also approve or reject it (checker).	`packages/shared/src/errors/same-actor-cannot-approve-own-change-error.ts`
`SIGNING_UNAVAILABLE`	503			`packages/runtime/src/errors/SigningUnavailableError.ts`
`STEP_UP_AUTHORIZATION_INVALID`	403	`approveApprovalIssuerChange`, `approveAuthorityGrantChange`, `approveBusinessSignalSourceChange`, `approveExternalConnectorChange`, `approvePolicyChange`, `rejectApprovalIssuerChange`, `rejectAuthorityGrantChange`, `rejectBusinessSignalSourceChange`, `rejectExternalConnectorChange`, `rejectPolicyChange`	The step-up authorization envelope is missing, invalid, expired, replayed, or does not match this request.	`packages/shared/src/errors/step-up-authorization-invalid-error.ts`
`TENANT_NOT_ALLOWED`	403			`packages/api/src/routes/execute.ts`
`VERIFICATION_FAILED`	404	`generateReceipt`, `replayTransaction`, `verifyTransaction`	Execution Trust Record not found.	`packages/runtime/src/errors/VerificationFailedError.ts`